

UNIVERSITÀ TELEMATICA PEGASO

Corso di laurea in
Informatica per le Aziende Digitali (L-31)

Insegnamento di
Project Work

**Sviluppo di un software di geolocaliz-
zazione culturale per condividere il
patrimonio librario degli utenti privati**

RELATORE:

inserire il nome del relatore

CANDIDATO:

Luigi Avitabile

0312501419

Il patrimonio librario posseduto da utenti privati rappresenta una risorsa culturale distribuita, spesso significativa ma difficilmente accessibile al di fuori della sfera personale del proprietario. In assenza di strumenti di catalogazione e pubblicazione, una collezione privata rimane non individuabile dall'esterno, e la distanza tra disponibilità fisica e accessibilità effettiva impedisce la scoperta di volumi potenzialmente utili a una comunità di lettori.

Questa tesi affronta il problema progettando e realizzando un prototipo web geolocalizzato per rendere catalogabili, individuabili e condivisibili patrimoni librari appartenenti a utenti privati. Il sistema consente di registrarsi, pubblicare libri con metadati bibliografici, renderli ricercabili tramite criteri testuali e geografici, visualizzarli su mappa interattiva e ricevere richieste di consultazione, prestito o informazione. Un aspetto centrale della soluzione riguarda la tutela della posizione: il progetto distingue tra coordinate precise, usate internamente, e coordinate pubbliche approssimate, utilizzate per la discovery territoriale, in modo da conciliare prossimità geografica e minimizzazione dell'esposizione dei dati sensibili.

L'architettura è basata su un monorepo modulare con Next.js, PostgreSQL/PostGIS, Prisma, Auth.js e MapLibre GL JS. La validazione ha combinato controlli statici, build di produzione e scenari funzionali manuali, confermando la correttezza dei flussi principali: autenticazione, catalogazione, geolocalizzazione, ricerca per prossimità e richieste tra utenti. I risultati dimostrano come un requisito di localizzazione possa essere integrato in una piattaforma di catalogazione senza esporre direttamente la posizione precisa degli utenti, fornendo un modello realizzabile e verificabile di discovery territoriale per risorse fisiche distribuite.

Indice

1	Introduzione	1
1.1	Contesto e motivazioni	1
1.2	Obiettivi del progetto	2
1.3	Metodologia e struttura dell’elaborato	3
2	Contesto applicativo e stato dell’arte	5
2.1	Digitalizzazione e condivisione del patrimonio librario	5
2.2	Sistemi geolocalizzati e Location-Based Services	6
2.3	Soluzioni esistenti e approcci comparabili	7
2.3.1	Catalogazione e gestione di collezioni personali	7
2.3.2	Condivisione fisica e dimensione territoriale	8
2.3.3	Analisi comparativa	8
2.4	Geolocalizzazione e tutela della privacy	9
2.5	Sintesi dello stato dell’arte e posizionamento del progetto	10
3	Analisi dei requisiti	11
3.1	Analisi del problema	11
3.2	Attori e casi d’uso	12
3.3	Requisiti funzionali	13
3.4	Requisiti non funzionali	16
3.5	Vincoli e assunzioni progettuali	18
4	Progettazione e architettura	19
4.1	Visione architetture del sistema	19
4.2	Stack tecnologico e criteri di scelta	20
4.3	Modello dei dati e persistenza	21
4.4	Progettazione della geolocalizzazione e tutela della posizione	22
4.5	Servizi esterni e strategie di fallback	23

5	Implementazione	25
5.1	Organizzazione del codice applicativo	25
5.2	Autenticazione, conferma email e profilo utente	26
5.3	Catalogazione libraria e gestione delle immagini	28
5.4	Catalogazione assistita tramite ISBN, OCR e metadati esterni	29
5.5	Geolocalizzazione, query spaziali e visualizzazione cartografica	30
5.6	Richieste tra utenti, dashboard e amministrazione	31
6	Testing e validazione	33
6.1	Strategia di verifica	33
6.1.1	Obiettivi della validazione	33
6.1.2	Livelli di verifica adottati	34
6.2	Verifica statica e qualità del codice	36
6.2.1	Type checking	36
6.2.2	Linting e convenzioni	37
6.2.3	Verifica della build	37
6.2.4	Test unitari	38
6.2.5	Test di integrazione	39
6.2.6	Smoke test end-to-end	40
6.2.7	Riproducibilità dell'ambiente	41
6.3	Validazione dei requisiti funzionali	41
6.3.1	Account e autenticazione	42
6.3.2	Gestione del patrimonio librario	43
6.3.3	Catalogazione assistita	44
6.3.4	Ricerca e discovery	46
6.3.5	Condivisione e richieste	46
6.4	Validazione della geolocalizzazione e della privacy	48
6.4.1	Geocoding e fallback	48
6.4.2	Approssimazione delle coordinate	49
6.4.3	Ricerca per prossimità	51
6.4.4	Visualizzazione cartografica	52
6.5	Tracciabilità e sintesi della validazione	53
7	Risultati e discussione	56
7.1	Risultati ottenuti	56
7.2	Discussione delle scelte progettuali	57

7.3	Limiti del prototipo	58
7.4	Sviluppi futuri	59
8	Conclusioni	61
8.1	Sintesi del lavoro svolto	61
8.2	Contributo progettuale	62
8.3	Considerazioni finali	62
	Bibliografia	65

Elenco delle tabelle

2.1	Confronto concettuale tra approcci comparabili	9
3.1	Casi d'uso principali del sistema	13
3.2	Requisiti funzionali del prototipo	14
3.3	Tracciabilità tra requisiti funzionali e casi d'uso	16
3.4	Requisiti non funzionali	16
4.1	Responsabilità architetturali principali	20
4.2	Entità principali del modello persistente	22
4.3	Servizi esterni e comportamento degradato	24
6.1	Livelli di verifica adottati	35
6.2	Esiti della verifica statica, di build e dei test automatizzati	37
6.3	Casi di test per account e autenticazione	42
6.4	Casi di test per la gestione del patrimonio librario	43
6.5	Casi di test per catalogazione assistita	45
6.6	Casi di test per ricerca e discovery	46
6.7	Casi di test per condivisione e richieste	47
6.8	Casi di test per geocoding e fallback	48
6.9	Casi di test per approssimazione e privacy delle coordinate	50
6.10	Casi di test per ricerca per prossimità	51
6.11	Casi di test per visualizzazione cartografica	52
6.12	Matrice di tracciabilità della validazione	53

API

Application Programming Interface.

CRUD

Create, Read, Update, Delete.

GDPR

General Data Protection Regulation.

ISBN

International Standard Book Number.

JWT

JSON Web Token.

LBS

Location-Based Service.

OCR

Optical Character Recognition.

RNF

Requisito Non Funzionale.

RF Requisito Funzionale.

SQL

Structured Query Language.

UC Use Case.

Capitolo 1

Introduzione

1.1 Contesto e motivazioni

Il patrimonio librario posseduto da utenti privati costituisce una risorsa culturale distribuita, spesso significativa dal punto di vista informativo ma difficilmente accessibile al di fuori della sfera personale del proprietario. Un libro fisico, infatti, può avere valore non soltanto per chi lo conserva, ma anche per altri utenti interessati alla consultazione, allo studio o al prestito. Tuttavia, il semplice possesso di una risorsa non ne implica la visibilità: in assenza di strumenti di catalogazione e pubblicazione, una collezione privata resta normalmente non individuabile dall'esterno.

Il problema affrontato in questa tesi nasce da questa distanza tra disponibilità fisica e accessibilità effettiva. Molti volumi presenti in collezioni personali potrebbero essere utili a una comunità di lettori, studenti o appassionati, ma non esiste necessariamente un meccanismo attraverso cui tali libri possano essere cercati, localizzati e richiesti. La questione non riguarda la digitalizzazione del contenuto dei libri, né la sostituzione delle biblioteche tradizionali, bensì la possibilità di rappresentare digitalmente l'esistenza di copie fisiche appartenenti a utenti privati e di renderle scopribili in modo controllato.

Gli strumenti software permettono di affrontare questo problema introducendo una rappresentazione strutturata delle collezioni: metadati bibliografici, stato di disponibilità, visibilità pubblica e canali di interazione tra utenti. Attraverso un catalogo digitale, il patrimonio librario privato può essere trasformato da insieme isolato di oggetti fisici a risorsa potenzialmente consultabile all'interno di una comunità. La ricerca testuale consente di individuare un volume per titolo, autore o altri attributi bibliografici; la gestione degli utenti e delle richieste permette invece di passare dalla

sola scoperta all'interazione con il proprietario.

In questo scenario assume particolare rilievo la dimensione geografica. I libri considerati dal progetto sono beni materiali: la possibilità concreta di consultarli o prenderli in prestito dipende anche dalla distanza tra chi possiede il volume e chi lo cerca. La prossimità territoriale diventa quindi un criterio rilevante di discovery. Tale impostazione riconduce il sistema alla categoria dei Location-Based Services, nei quali l'informazione geografica contribuisce alla selezione o all'ordinamento dei risultati. Al tempo stesso, l'utilizzo della posizione introduce un vincolo progettuale rilevante: la piattaforma deve offrire utilità geografica senza esporre in modo indiscriminato la posizione precisa degli utenti, coerentemente con i rischi discussi nella letteratura sui servizi basati sulla localizzazione [1, 2].

Culturando nasce come prototipo web geolocalizzato per affrontare questo insieme di esigenze. Il sistema consente a utenti privati di registrarsi, rappresentare digitalmente i propri libri, pubblicarli in un catalogo, renderli ricercabili tramite criteri bibliografici e geografici, e ricevere richieste di consultazione, prestito o informazione. Una parte centrale della soluzione riguarda la tutela della posizione: il progetto distingue tra coordinate precise, usate internamente quando necessario, e coordinate pubbliche approssimate, utilizzate per la visualizzazione e la ricerca geografica. In questo modo il prototipo mira a conciliare discovery territoriale e limitazione dell'esposizione dei dati sensibili.

1.2 Obiettivi del progetto

L'obiettivo generale del progetto è progettare e realizzare un prototipo funzionale di piattaforma web geolocalizzata per rendere catalogabili, individuabili e condivisibili patrimoni librari appartenenti a utenti privati. Tale obiettivo traduce il tema del lavoro in un sistema software concreto, nel quale la gestione del libro fisico viene collegata alla ricerca digitale, alla prossimità geografica e all'interazione tra utenti.

Il primo obiettivo funzionale riguarda la catalogazione. La piattaforma deve permettere a un utente registrato di costruire una rappresentazione digitale della propria collezione libraria, associando a ciascun libro metadati essenziali quali titolo, autore, descrizione, ISBN, lingua, editore, categorie, immagini e stato di disponibilità. La catalogazione non ha lo scopo di riprodurre il contenuto dell'opera, ma di descrivere il bene fisico in modo sufficiente a renderlo identificabile e gestibile.

Il secondo obiettivo riguarda la discovery bibliografica. I libri pubblicati devono

poter essere individuati da altri utenti attraverso ricerche testuali e criteri di filtro coerenti con il dominio applicativo. La ricerca rappresenta il passaggio fondamentale tra catalogazione individuale e valore comunitario: senza meccanismi di individuazione, la collezione rimarrebbe una semplice lista privata.

Il terzo obiettivo è l'integrazione della prossimità geografica. Poiché la consultazione o il prestito di un libro fisico richiedono una relazione spaziale tra utenti, il sistema deve consentire la ricerca di libri disponibili in un'area determinata. Questo implica la gestione di indirizzi, coordinate, raggi di ricerca e ordinamento per distanza, mantenendo però separata la posizione operativa interna dalla posizione esposta pubblicamente.

Il quarto obiettivo riguarda la condivisione controllata. La piattaforma deve supportare l'invio e la gestione di richieste tra utenti, evitando che la pubblicazione di un libro comporti automaticamente l'esposizione di recapiti personali. Il proprietario deve poter ricevere una richiesta, accettarla o rifiutarla, mentre l'utente interessato deve poter seguire lo stato della propria interazione.

Accanto agli obiettivi funzionali, il progetto considera alcuni obiettivi trasversali. La privacy della posizione è il vincolo principale, perché la geolocalizzazione aumenta l'utilità del sistema ma può introdurre rischi se progettata in modo ingenuo. Sono inoltre rilevanti la sicurezza dell'autenticazione, la separazione delle responsabilità tra le parti del sistema, la manutenibilità del codice, la riproducibilità dell'ambiente di sviluppo e la possibilità di estendere il prototipo con servizi opzionali, come catalogazione assistita, provider esterni di geocoding o meccanismi di archiviazione delle immagini.

1.3 Metodologia e struttura dell'elaborato

Il lavoro è stato sviluppato seguendo un percorso progettuale incrementale. In primo luogo è stato analizzato il problema della scarsa visibilità del patrimonio librario privato e sono stati individuati gli attori principali del sistema. Successivamente sono stati definiti i requisiti funzionali e non funzionali, con particolare attenzione alla relazione tra catalogazione, ricerca, geolocalizzazione e tutela della posizione. La fase di progettazione ha quindi tradotto tali requisiti in un'architettura applicativa e in un modello dei dati coerenti con il dominio. Infine, il prototipo è stato implementato e predisposto per una validazione funzionale basata su casi d'uso rappresentativi.

La tesi è organizzata in otto capitoli. Il Capitolo 2 introduce il contesto applicativo, discutendo digitalizzazione del patrimonio librario, discovery, servizi

geolocalizzati e privacy della posizione. Il Capitolo 3 formalizza l'analisi del problema attraverso attori, casi d'uso, requisiti funzionali, requisiti non funzionali e vincoli progettuali. Il Capitolo 4 descrive le scelte architettureali, il modello dei dati, la gestione della geolocalizzazione e l'integrazione con servizi esterni. Il Capitolo 5 presenta gli aspetti implementativi più rilevanti del prototipo, collegando le scelte tecniche ai requisiti individuati.

Il Capitolo 6 è dedicato alla verifica del sistema mediante test funzionali e scenari manuali ripetibili. Il Capitolo 7 discute i risultati ottenuti, i limiti del prototipo e le possibili evoluzioni future. Il Capitolo 8 conclude il lavoro riassumendo il contributo progettuale e valutando il raggiungimento degli obiettivi iniziali.

Capitolo 2

Contesto applicativo e stato dell'arte

2.1 Digitalizzazione e condivisione del patrimonio librario

Nel dominio considerato da questa tesi, la digitalizzazione non coincide con la trasformazione del contenuto dei libri in formato elettronico. Il progetto non mira a costruire una biblioteca digitale di testi consultabili online, né a sostituire il libro fisico con un ebook. L'oggetto principale resta il volume materiale posseduto da un utente privato; ciò che viene digitalizzato è la sua rappresentazione informativa, cioè l'insieme dei dati necessari per descriverlo, organizzarne la disponibilità e renderlo individuabile.

La rappresentazione digitale di una collezione fisica richiede metadati sufficientemente strutturati. Titolo, autore, ISBN, editore, lingua, anno di pubblicazione, categoria, descrizione e stato di disponibilità sono esempi di attributi che permettono al sistema di distinguere un libro da un altro e di supportare operazioni di ricerca. A questi dati possono aggiungersi immagini o copertine, utili per il riconoscimento visivo, ma non essenziali quanto i metadati bibliografici per la costruzione di un catalogo interrogabile.

La sola catalogazione, tuttavia, non garantisce accessibilità. Una collezione può essere rappresentata digitalmente ma rimanere isolata se non viene indicizzata, pubblicata o collegata a meccanismi di ricerca. Nel passaggio da archivio personale a risorsa condivisibile diventa quindi centrale il concetto di discovery: l'utente interessato deve poter formulare una ricerca, ottenere risultati pertinenti e comprendere se un

determinato libro sia disponibile presso un altro soggetto. In questo senso, la meta-datazione è una condizione necessaria ma non sufficiente; occorre anche un modello applicativo che renda tali metadati utilizzabili in un contesto comunitario.

La condivisione del patrimonio librario privato introduce un ulteriore livello rispetto alla gestione individuale della collezione. Il catalogo personale diventa parte di un ambiente in cui altri utenti possono scoprire i libri, consultarne le informazioni e avviare una richiesta. Questo modello si avvicina al concetto di catalogazione sociale, nel quale la descrizione delle risorse non rimane confinata al singolo proprietario, ma contribuisce alla costruzione di un insieme informativo utile alla comunità. Per *Culturando*, tale passaggio è fondamentale: il valore del sistema non risiede soltanto nella gestione privata della biblioteca personale, ma nella possibilità di trasformare collezioni distribuite in risorse visibili e richiedibili.

2.2 Sistemi geolocalizzati e Location-Based Services

I Location-Based Services sono sistemi nei quali l'informazione geografica dell'utente, di una risorsa o di un evento contribuisce al funzionamento del servizio. La posizione può essere usata come input per filtrare risultati, ordinarli, visualizzarli su una mappa o adattare l'esperienza al contesto spaziale. Nel caso di una piattaforma libraria, la posizione non descrive soltanto un punto sulla carta, ma fornisce un'indicazione sulla fattibilità pratica di una consultazione o di un prestito.

Un modello concettuale generale di servizio geolocalizzato comprende almeno quattro elementi: un utente o una richiesta, una posizione di riferimento, un insieme di risorse localizzate e una logica di elaborazione spaziale. Applicando tale modello al dominio di *Culturando*, l'utente specifica un luogo o un'area di interesse; il sistema confronta tale informazione con la localizzazione pubblica dei libri disponibili; infine restituisce un insieme di risultati coerenti con la distanza o il raggio selezionato.

La prossimità geografica introduce criteri diversi dalla ricerca testuale. In una ricerca bibliografica tradizionale, la pertinenza dipende soprattutto dalla corrispondenza tra query e metadati. In una ricerca spaziale, invece, assumono rilievo la distanza, il raggio massimo, la posizione di origine e la distribuzione territoriale delle risorse. Il risultato non è semplicemente il libro che corrisponde meglio a una stringa, ma il libro che soddisfa determinati vincoli informativi e si trova entro un'area utile per l'utente.

Nel progetto questa dimensione è motivata dalla natura fisica dei libri. A differenza di una risorsa interamente digitale, un volume cartaceo deve essere recuperato, consultato o scambiato in un luogo. La distanza tra proprietario e richiedente non è quindi un dettaglio accessorio, ma una variabile che può determinare la praticabilità dell'interazione. Il ruolo della geolocalizzazione consiste nel rendere esplicita questa variabile, trasformandola in un criterio di discovery controllato.

2.3 Soluzioni esistenti e approcci comparabili

Il confronto con soluzioni esistenti non viene impostato come analisi commerciale di concorrenti, ma come osservazione di approcci applicativi affini. Le piattaforme considerate evidenziano funzionalità rilevanti per il problema trattato: catalogazione personale, pubblicazione di collezioni, interazione comunitaria, circolazione fisica dei libri e uso della dimensione territoriale.

2.3.1 Catalogazione e gestione di collezioni personali

LibraryThing rappresenta un esempio noto di piattaforma orientata alla catalogazione personale e alla dimensione comunitaria del libro [3]. L'utente può organizzare la propria collezione, arricchirla con metadati e partecipare a un ambiente sociale costruito intorno ai libri. Un elemento rilevante per questa tesi è la presenza di funzionalità locali, come LibraryThing Local, che collegano il dominio librario a luoghi fisici quali biblioteche, librerie ed eventi [4]. La dimensione territoriale è quindi presente, ma è principalmente associata a luoghi culturali o commerciali collegati al libro, non alla localizzazione di singoli volumi posseduti da utenti privati.

Libib si colloca invece nell'ambito della gestione di cataloghi personali e domestici [5]. La piattaforma permette di organizzare collezioni, utilizzare identificatori come l'ISBN e pubblicare cataloghi consultabili. Il suo interesse, rispetto a *Culturando*, riguarda il tema della catalogazione privata resa accessibile tramite strumenti digitali. Tuttavia, il modello concettuale resta centrato soprattutto sulla gestione e sulla pubblicazione del catalogo, mentre la ricerca per prossimità geografica di copie fisiche appartenenti ad altri utenti non costituisce l'asse principale del confronto.

2.3.2 Condivisione fisica e dimensione territoriale

BookCrossing adotta un modello differente, basato sulla circolazione fisica del libro [6]. Il volume viene registrato, identificato e poi rilasciato affinché altri utenti possano trovarlo, leggerlo e rimetterlo in circolazione. La componente comunitaria e geografica è rilevante, perché il percorso del libro può essere seguito attraverso luoghi e passaggi successivi. La differenza rispetto a *Culturando* è sostanziale: in BookCrossing il libro è pensato per muoversi e cambiare temporaneamente possessore nel contesto di un rilascio pubblico; in *Culturando* il proprietario conserva il controllo della copia e decide se renderla disponibile per consultazione, prestito o semplice richiesta informativa.

Little Free Library propone un altro modello territoriale, fondato su punti fisici di scambio distribuiti nello spazio urbano o locale [7]. La mappa permette di individuare piccole biblioteche comunitarie vicine, intese come luoghi nei quali i libri possono essere depositati o prelevati. Anche in questo caso la geolocalizzazione è centrale, ma l'oggetto localizzato è il punto di scambio, non il singolo libro appartenente a una collezione privata digitale. Il sistema valorizza la prossimità e la comunità, ma attraverso infrastrutture fisiche condivise anziché attraverso cataloghi personali geolocalizzati.

2.3.3 Analisi comparativa

Le soluzioni considerate mostrano che catalogazione, comunità e territorio possono combinarsi in modi diversi. LibraryThing enfatizza la catalogazione personale e la dimensione sociale; Libib si concentra sulla gestione e pubblicazione di collezioni; BookCrossing valorizza la circolazione fisica del libro; Little Free Library organizza la condivisione intorno a punti territoriali di scambio. *Culturando* si colloca all'intersezione di questi approcci, ma adotta un oggetto geografico diverso: non il luogo culturale, non la biblioteca comunitaria e non il percorso di un libro rilasciato, bensì la disponibilità approssimata di un volume appartenente a un utente privato.

Tabella 2.1: Confronto concettuale tra approcci comparabili

Soluzione	Approccio principale	Differenza rispetto a <i>Culturando</i>
LibraryThing	Catalogazione personale, comunità e luoghi collegati al dominio librario	La componente territoriale riguarda soprattutto luoghi ed eventi, non la disponibilità approssimata di singoli libri privati
Libib	Gestione e pubblicazione di cataloghi personali	La prossimità geografica non è il criterio centrale di discovery nel modello considerato
BookCrossing	Rilascio, circolazione e tracciamento del libro fisico	Il libro circola nello spazio; in <i>Culturando</i> resta controllato dal proprietario
Little Free Library	Punti fisici di scambio consultabili su mappa	L'oggetto geolocalizzato è la biblioteca comunitaria, non il libro di un singolo utente
<i>Culturando</i>	Catalogazione privata, ricerca bibliografica e discovery geografica	L'oggetto geolocalizzato è la disponibilità approssimata di un libro privato

La tabella non intende stabilire una superiorità tra sistemi, ma chiarire il posizionamento progettuale. *Culturando* integra catalogazione privata, discovery testuale, ricerca per prossimità e gestione delle richieste, mantenendo come vincolo specifico la protezione della posizione precisa dell'utente.

2.4 Geolocalizzazione e tutela della privacy

La posizione geografica è un'informazione delicata perché può contribuire all'identificazione di abitudini, luoghi frequentati o contesti personali. In un sistema location-aware, la localizzazione non è soltanto un dato tecnico utile al calcolo della distanza: può diventare un elemento sensibile se associato a un profilo utente, a una risorsa posseduta o a una sequenza di interazioni. Per questo motivo, la progettazione di servizi geolocalizzati richiede attenzione alla minimizzazione dell'esposizione e al controllo della granularità del dato.

La letteratura sulla location privacy evidenzia il compromesso tra utilità del servizio e rischio di esposizione [1, 2, 8]. Coordinate più precise consentono risultati più accurati, ma aumentano anche la possibilità di inferire informazioni personali. Al contrario, una localizzazione più approssimata riduce il dettaglio disponibile e quindi può limitare il rischio, pur introducendo una perdita di precisione nel servizio. La scelta progettuale deve quindi bilanciare accuratezza, usabilità e tutela dell'utente.

Tra le strategie generali adottabili rientrano la minimizzazione dei dati raccolti, la riduzione della granularità spaziale, l'approssimazione delle coordinate, lo spatial cloaking e il controllo da parte dell'utente sulle informazioni rese visibili. Non tutte queste tecniche sono necessariamente appropriate per ogni sistema. Nel caso di *Culturando*, l'esigenza principale è consentire la discovery entro un'area geografica senza mostrare la posizione esatta del proprietario del libro. La soluzione concreta adottata dal prototipo verrà descritta nel Capitolo 4, dove la tutela della posizione è collegata al modello dei dati e alle query spaziali.

2.5 Sintesi dello stato dell'arte e posizionamento del progetto

L'analisi svolta mostra che il dominio della condivisione libraria può essere affrontato da prospettive diverse. Alcune piattaforme privilegiano la catalogazione e la gestione personale della collezione; altre valorizzano la dimensione comunitaria, la circolazione fisica del libro o la presenza di punti territoriali di scambio. La geolocalizzazione compare in più forme, ma non sempre è associata alla disponibilità approssimata di singoli libri appartenenti a utenti privati.

Il posizionamento di *Culturando* deriva dall'intersezione tra cinque elementi: catalogazione privata, pubblicazione controllata, discovery bibliografica, ricerca geografica e tutela della posizione. Il progetto non intende sostituire le piattaforme esistenti, ma esplorare un modello applicativo specifico nel quale il patrimonio librario privato diventa ricercabile in base sia ai metadati sia alla prossimità, senza rendere pubbliche coordinate precise. Questa analisi fornisce la base per la formalizzazione dei requisiti nel Capitolo 3.

Capitolo 3

Analisi dei requisiti

3.1 Analisi del problema

Il problema introdotto nei capitoli precedenti può essere ricondotto alla gestione di una risorsa fisica distribuita: il libro appartenente a una collezione privata. Tale risorsa possiede caratteristiche bibliografiche, uno stato di disponibilità, un proprietario e una collocazione territoriale. Il sistema deve quindi rappresentare il libro come entità informativa, consentirne la pubblicazione controllata e permettere ad altri utenti di individuarlo attraverso criteri testuali e geografici.

La difficoltà principale non consiste soltanto nel salvare una scheda libro, ma nel collegare tre esigenze: catalogazione, discovery e interazione. La catalogazione permette al proprietario di descrivere la propria collezione; la discovery consente a un altro utente di trovare libri pertinenti; l'interazione trasforma il risultato della ricerca in una richiesta gestibile. In assenza di uno di questi elementi, il sistema resterebbe incompleto: un catalogo non ricercabile avrebbe utilità limitata, mentre una ricerca priva di canale di richiesta non supporterebbe realmente la condivisione.

La dimensione geografica aggiunge un vincolo specifico. Poiché i libri sono beni fisici, la distanza tra proprietario e utente interessato può determinare la fattibilità della consultazione o del prestito. Tuttavia, la posizione associata a un libro privato può rivelare informazioni personali sul proprietario. Il sistema deve quindi trattare la localizzazione come requisito funzionale per la ricerca e, allo stesso tempo, come requisito non funzionale di privacy.

L'analisi dei requisiti considera il prototipo come applicazione web destinata a utenti autenticati e visitatori. Il catalogo pubblico è consultabile come vetrina del patrimonio disponibile, mentre le funzionalità che producono effetti persistenti,

come pubblicazione di libri, modifica del profilo, ricerca geografica protetta e richieste, richiedono autenticazione. Tale distinzione permette di separare la fase di esplorazione pubblica dalla gestione personale e dalle interazioni tra utenti.

3.2 Attori e casi d'uso

Gli attori individuati derivano dai ruoli effettivamente presenti nel prototipo e dalle responsabilità applicative emerse dall'analisi del dominio.

- **Visitatore non autenticato:** può accedere alle pagine pubbliche e consultare il catalogo, ma non può creare contenuti, inviare richieste o accedere alle aree protette.
- **Utente registrato:** dispone di un account verificato, può accedere alla dashboard, gestire il profilo, pubblicare libri, cercare disponibilità geografiche e inviare richieste.
- **Proprietario di libri:** è un utente registrato che possiede una o più schede libro e può modificarle, eliminarle e gestire le richieste ricevute.
- **Amministratore:** è un utente con ruolo specifico che può accedere alla dashboard amministrativa e consultare metriche aggregate del prototipo.

I principali casi d'uso sono sintetizzati in table 3.1. Gli identificativi sono mantenuti stabili per permettere il collegamento con i requisiti funzionali e con la successiva matrice di validazione.

Tabella 3.1: Casi d'uso principali del sistema

ID	Attore	Descrizione
UC-01	Visitatore	Registrarsi e confermare l'indirizzo email tramite link di verifica.
UC-02	Utente registrato	Autenticarsi e accedere alle aree protette dell'applicazione.
UC-03	Utente registrato	Modificare profilo, dati descrittivi, domicilio e visibilità pubblica.
UC-04	Proprietario	Pubblicare un libro con metadati, immagini, stato, visibilità e posizione.
UC-05	Proprietario	Modificare o eliminare un libro posseduto.
UC-06	Visitatore o utente	Consultare il catalogo pubblico e cercare libri per metadati bibliografici.
UC-07	Utente registrato	Cercare libri disponibili vicino a un indirizzo o a un'area geografica.
UC-08	Utente registrato	Visualizzare risultati geografici in forma testuale e cartografica.
UC-09	Utente registrato	Inviare una richiesta di consultazione, prestito o informazione per un libro pubblico.
UC-10	Proprietario	Accettare o rifiutare richieste ricevute sui propri libri.
UC-11	Utente registrato	Consultare le richieste inviate e cancellare quelle ancora in attesa.
UC-12	Utente registrato	Consultare dashboard personale, statistiche e libri pubblicati.
UC-13	Amministratore	Accedere alla dashboard amministrativa con statistiche aggregate.

3.3 Requisiti funzionali

I requisiti funzionali descrivono i comportamenti osservabili che il prototipo deve supportare. La numerazione RF-01, RF-02 e successive viene utilizzata come riferimento stabile per la progettazione, l'implementazione e la validazione.

Tabella 3.2: Requisiti funzionali del prototipo

ID	Area	Requisito
RF-01	Account	Il sistema deve consentire la registrazione di un nuovo utente tramite nome, email e password.
RF-02	Verifica email	Il sistema deve generare un token di verifica email e consentire l'accesso solo agli utenti con account confermato.
RF-03	Autenticazione	Il sistema deve permettere il login tramite credenziali e proteggere dashboard, profilo, impostazioni, ricerca geografica e funzioni operative.
RF-04	Profilo	L'utente autenticato deve poter modificare i dati del profilo, inclusi nome, nickname, avatar, biografia, domicilio e visibilità pubblica.
RF-05	Catalogazione	Il proprietario deve poter creare una scheda libro con metadati bibliografici, stato di disponibilità, visibilità, condizione fisica e indirizzo.
RF-06	Immagini	Il sistema deve supportare immagini associate al libro, distinguendo immagine principale, immagini caricate dall'utente, immagini esterne e thumbnail quando disponibili.
RF-07	Modifica ed eliminazione	Il proprietario deve poter modificare ed eliminare esclusivamente i libri di cui è titolare.
RF-08	Catalogazione assistita	Il sistema deve supportare funzioni di assistenza alla compilazione tramite ISBN, recupero metadati e OCR opzionale, lasciando all'utente la conferma dei dati proposti.

ID	Area	Requisito
RF-09	Catalogo pubblico	Il sistema deve mostrare un catalogo consultabile dei libri, con dettaglio della scheda, metadati, immagini, disponibilità e conteggio delle visualizzazioni.
RF-10	Ricerca testuale	Il sistema deve consentire la ricerca dei libri per campi bibliografici e descrittivi, tra cui titolo, autore, ISBN, editore, città, categoria e descrizione.
RF-11	Ricerca geografica	L'utente autenticato deve poter cercare libri disponibili entro un raggio configurabile a partire da un indirizzo o da un'area.
RF-12	Mappa	Il sistema deve visualizzare i risultati geografici su mappa interattiva, con marker, cluster, popup informativi e modalità di visualizzazione 2D/3D.
RF-13	Richieste	L'utente autenticato deve poter inviare richieste di tipo consultazione, prestito o informazione per libri pubblici e disponibili di altri utenti.
RF-14	Gestione richieste	Il proprietario deve poter accettare o rifiutare richieste ricevute, mentre il richiedente deve poter cancellare richieste ancora in stato pending.
RF-15	Regole di proprietà	Il sistema deve impedire la creazione di richieste verso libri appartenenti allo stesso utente richiedente.
RF-16	Dashboard personale	L'utente autenticato deve poter consultare un'area personale con libri pubblicati, richieste e statistiche d'uso.
RF-17	Amministrazione	Il sistema deve consentire l'accesso a una dashboard amministrativa solo agli utenti con ruolo admin.

ID	Area	Requisito
RF-18	Impostazioni	L'utente deve poter modificare preferenze applicative supportate dal prototipo, come la lingua dell'interfaccia.

La tracciabilità iniziale tra requisiti e casi d'uso è riportata in table 3.3. Tale matrice non sostituisce la validazione, ma permette di verificare che ogni requisito sia collegato ad almeno uno scenario osservabile.

Tabella 3.3: Tracciabilità tra requisiti funzionali e casi d'uso

Requisito	Casi d'uso collegati
RF-01, RF-02, RF-03	UC-01, UC-02
RF-04, RF-18	UC-03
RF-05, RF-06, RF-07, RF-08	UC-04, UC-05
RF-09, RF-10	UC-06
RF-11, RF-12	UC-07, UC-08
RF-13, RF-14, RF-15	UC-09, UC-10, UC-11
RF-16	UC-12
RF-17	UC-13

3.4 Requisiti non funzionali

I requisiti non funzionali precisano qualità, vincoli e proprietà trasversali che influenzano la progettazione. In questo progetto assumono particolare importanza privacy, sicurezza e manutenibilità, perché la piattaforma gestisce account utente, contenuti pubblicabili e dati di posizione.

Tabella 3.4: Requisiti non funzionali

ID	Categoria	Requisito
RNF-01	Privacy geografica	Il sistema non deve esporre pubblicamente le coordinate precise associate alla posizione di un libro.

ID	Categoria	Requisito
RNF-02	Minimizzazione	Le informazioni geografiche pubbliche devono essere ridotte alla granularità necessaria per discovery e mappa.
RNF-03	Sicurezza account	Le password devono essere memorizzate in forma non reversibile e il login deve rifiutare account non verificati.
RNF-04	Autorizzazione	Le operazioni su profilo, libri, richieste e amministrazione devono verificare identità, proprietà o ruolo dell'utente.
RNF-05	Validazione input	I dati inseriti nei form devono essere validati prima della persistenza, con messaggi coerenti per l'utente.
RNF-06	Modularità	Il sistema deve mantenere separate route, componenti di dominio, componenti UI generici e pacchetti riutilizzabili.
RNF-07	Manutenibilità	Le funzionalità devono essere organizzate per feature e supportare evoluzioni successive senza duplicazione non necessaria.
RNF-08	Riproducibilità	L'ambiente locale deve essere avviabile tramite configurazione e comandi documentati, includendo database e servizi opzionali.
RNF-09	Degrado controllato	Le integrazioni esterne opzionali devono prevedere fallback o comportamento degradato quando non configurate.
RNF-10	Usabilità responsiva	Le pagine principali, il catalogo, la dashboard e la mappa devono restare utilizzabili su viewport desktop e mobile.
RNF-11	Internazionalizzazione	I testi applicativi devono essere centralizzati in dizionari riutilizzabili quando presenti.

ID	Categoria	Requisito
RNF-12	Osservabilità funzionale	Il prototipo deve esporre statistiche essenziali su libri, richieste e visualizzazioni, utili alla valutazione dell'uso del sistema.

3.5 Vincoli e assunzioni progettuali

Il primo vincolo è la natura prototipale del progetto. *Culturando* è sviluppato come elaborato universitario e come prototipo funzionale: deve dimostrare la fattibilità del modello applicativo, ma non pretende di coprire tutti i requisiti operativi di una piattaforma pubblica in produzione. Aspetti come moderazione avanzata, notifiche complete, messaggistica interna estesa o procedure amministrative complesse possono essere considerati sviluppi successivi.

Il secondo vincolo riguarda l'ambiente di esecuzione. Il sistema è progettato per essere riproducibile localmente, con database relazionale e supporto geospaziale avviabili tramite l'ambiente di sviluppo. Alcuni servizi esterni sono configurabili ma opzionali: geocoding, OCR, storage delle immagini e invio email possono dipendere da credenziali o provider specifici. L'analisi dei requisiti considera quindi anche il comportamento in modalità degradata o locale, purché coerente con gli obiettivi del prototipo.

Un ulteriore vincolo riguarda la privacy della posizione. Il sistema assume che la ricerca geografica debba essere utile senza pubblicare coordinate precise. Per questa ragione, i requisiti distinguono tra dato geografico interno e dato geografico pubblico. La progettazione tecnica di questa separazione è discussa nel Capitolo 4, mentre la sua validazione viene trattata nel Capitolo 6.

Infine, la validazione prevista per il prototipo combina controlli statici, build, test unitari automatizzati sulla logica pura, test di integrazione mirati ai vincoli di autorizzazione, uno smoke test end-to-end sul percorso critico e scenari manuali ripetibili. Non viene assunta la presenza di una suite end-to-end estesa, né di una copertura automatica dell'intero codice applicativo o di tutti i flussi funzionali. Questa scelta costituisce un limite metodologico, ma è coerente con l'obiettivo di verificare i flussi principali del sistema in relazione ai requisiti definiti in questo capitolo.

Capitolo 4

Progettazione e architettura

4.1 Visione architetturale del sistema

La progettazione di *Culturando* deriva dai requisiti individuati nel Capitolo 3. Il sistema deve gestire utenti, libri, immagini, posizioni, richieste e statistiche; deve inoltre distinguere tra funzionalità pubbliche, come la consultazione del catalogo, e funzionalità protette, come la pubblicazione dei libri, la ricerca geografica e la gestione delle richieste. Per questo motivo l'architettura è stata organizzata come applicazione web modulare, con separazione tra interfaccia, logica di dominio, persistenza e funzionalità riutilizzabili.

Il repository adotta una struttura monorepo basata su **Nx** e **pnpm**. La scelta del monorepo non è motivata dalla dimensione attuale del prototipo, ma dall'esigenza di mantenere separabili le responsabilità e di predisporre il progetto a evoluzioni future. L'applicazione web principale si trova in **apps/web**, mentre i pacchetti condivisi sono collocati in **packages**. In questo modo componenti specifici della web app, come pagine, form e interazioni, restano separati da funzioni riutilizzabili, come accesso al database, tipi di dominio, geocoding, traduzioni e catalogazione assistita.

La struttura architetturale distingue tre livelli principali. Il primo livello è quello applicativo, rappresentato dalle route **Next.js** e dai componenti React specifici della web app. Il secondo livello è quello delle feature, nel quale sono isolate le funzionalità di dominio: autenticazione, libri, profilo, richieste, dashboard, amministrazione, geolocalizzazione e catalogazione assistita. Il terzo livello è quello dei pacchetti condivisi, che contiene codice non legato a una singola pagina o interazione. Questa separazione riduce l'accoppiamento tra le parti del sistema e rende più chiaro dove collocare nuove funzionalità.

Tabella 4.1: Responsabilità architetture principali

Area	Responsabilità
Route web	Routing, layout, pagine e protezione server-side delle route.
Feature web	Logica applicativa organizzata per funzionalità: form, azioni server, validazioni e componenti di dominio.
Componenti UI	Componenti grafici generici, privi di conoscenza del dominio applicativo.
Database	Schema Prisma , client database e accesso alla persistenza.
Geolocalizzazione	Geocoding, normalizzazione e approssimazione delle coordinate.
Catalogazione assistita	Supporto a ISBN, metadati bibliografici e OCR opzionale.
Traduzioni	Dizionari e funzioni pure per l'internazionalizzazione.

Questa organizzazione permette di collegare ogni scelta architetture a un requisito. La protezione delle route supporta i requisiti di autenticazione e autorizzazione; il pacchetto geografico isola le decisioni legate alla privacy della posizione; il pacchetto database centralizza il modello persistente; i dizionari condivisi riducono la duplicazione dei testi e preparano il sistema a ulteriori interfacce.

4.2 Stack tecnologico e criteri di scelta

La web app è realizzata con **Next.js**, **React** e **TypeScript**. **Next.js** è utilizzato per il routing tramite App Router, per la composizione delle pagine e per l'esecuzione server-side delle azioni che richiedono accesso alla sessione o al database [9]. **React** fornisce il modello a componenti per costruire interfacce riutilizzabili, mentre **TypeScript** introduce tipizzazione statica e migliora la robustezza del codice applicativo.

La gestione del workspace è affidata a **Nx** e **pnpm** [10]. **Nx** consente di organizzare applicazioni e pacchetti all'interno dello stesso repository, mantenendo confini architetture espliciti. **pnpm** gestisce le dipendenze del workspace e supporta la struttura a pacchetti dichiarata nel file **pnpm-workspace.yaml**. Questa combinazione è coerente con il requisito di modularità e con l'obiettivo di mantenere riutilizzabili le parti non specifiche della web app.

La persistenza utilizza **PostgreSQL** con estensione **PostGIS**, gestita tramite **Prisma** [11, 12, 13]. **PostgreSQL** è adatto al dominio relazionale del progetto, nel quale utenti, libri, immagini, posizioni, statistiche e richieste sono entità collegate.

PostGIS risponde al requisito di ricerca geografica, perché abilita operazioni spaziali come selezione entro un raggio e calcolo della distanza. **Prisma** fornisce lo schema applicativo e il client per le operazioni relazionali, mentre le query spaziali più specifiche vengono eseguite tramite SQL grezzo dove necessario.

L'autenticazione è basata su **Auth.js** con provider Credentials e sessione JWT [14]. Questa scelta permette di collegare il login agli utenti persistenti nel database, verificare la password lato server e arricchire la sessione con identificativo, ruolo e preferenze necessarie all'interfaccia. La mappa interattiva utilizza **MapLibre GL JS**, libreria WebGL per la visualizzazione cartografica con marker, cluster, popup e modalità 2D/3D [15]. Le immagini vengono processate con **Sharp**, usato per generare thumbnail WebP, mentre i servizi esterni opzionali sono incapsulati dietro adapter o azioni server.

4.3 Modello dei dati e persistenza

Il modello dei dati è definito nello schema **Prisma** del pacchetto **packages/db**. La progettazione segue le entità centrali del dominio: utente, libro, posizione, immagini, statistiche e richieste. La separazione in modelli distinti evita di concentrare informazioni eterogenee in un'unica tabella e permette di applicare vincoli specifici, come l'unicità del nickname, l'associazione uno-a-uno tra libro e posizione, e le relazioni con cancellazione a cascata per le entità dipendenti.

Il modello **User** rappresenta l'account applicativo. Oltre a email, nome e password hash, contiene il ruolo, la preferenza di saluto, i dati descrittivi del profilo, il domicilio e la visibilità pubblica del profilo. Il campo **emailVerifiedAt** è rilevante per il requisito di verifica account: un utente registrato ma non verificato non può autenticarsi. I token di conferma sono modellati separatamente in **EmailVerificationToken**, che memorizza l'hash del token e la scadenza.

Il modello **Book** descrive la scheda del libro appartenente a un utente. Contiene metadati bibliografici, stato di disponibilità, visibilità e condizione fisica. La posizione è separata nel modello **BookLocation**, che conserva sia coordinate precise sia coordinate pubbliche approssimate. Le immagini sono rappresentate da **BookImage**, con distinzione tra origine dell'immagine, immagine principale e thumbnail. Il modello **BookStats** raccoglie il conteggio delle visualizzazioni, evitando di sovraccaricare la scheda libro con dati di natura statistica.

Le interazioni tra utenti sono modellate tramite **LoanRequest**. Ogni richiesta

collega un libro, un richiedente e un proprietario; include inoltre il tipo di richiesta e uno stato. Gli stati previsti sono **pending**, **accepted**, **rejected**, **cancelled** e **completed**. Questa modellazione consente di verificare le regole di proprietà e autorizzazione: il richiedente non deve coincidere con il proprietario, il proprietario può gestire le richieste ricevute e il richiedente può cancellare quelle ancora in attesa.

Tabella 4.2: Entità principali del modello persistente

Entità	Ruolo nel sistema
User	Account, profilo, ruolo, preferenze e dati descrittivi dell'utente.
Token email	Token temporaneo per la conferma dell'indirizzo email.
Book	Scheda bibliografica e operativa di un libro posseduto da un utente.
BookLocation	Posizione del libro, con separazione tra coordinate precise e pubbliche.
BookImage	Immagini del libro, origine, immagine principale e thumbnail.
BookStats	Statistiche essenziali, in particolare numero di visualizzazioni.
LoanRequest	Richieste di consultazione, prestito o informazione tra utenti.
RateLimitBucket	Tabella tecnica, esterna al dominio: contatori per la limitazione dei tentativi su accesso, registrazione e richieste.

4.4 Progettazione della geolocalizzazione e tutela della posizione

La geolocalizzazione è progettata come funzione di discovery e non come esposizione dell'indirizzo preciso dell'utente. Quando un libro viene creato o modificato, l'indirizzo inserito viene inviato al livello geografico, che tenta di ottenere coordinate tramite geocoding. Il risultato contiene due rappresentazioni: le coordinate precise, conservate internamente quando disponibili, e le coordinate pubbliche approssimate. Nel codice corrente l'approssimazione avviene arrotondando latitudine e longitudine a tre cifre decimali.

La separazione tra coordinate precise e pubbliche risponde direttamente ai requisiti RNF-01 e RNF-02. Le coordinate precise possono essere utili come dato interno, ma non devono diventare l'informazione usata per la visualizzazione pubblica o per la discovery. La mappa e la ricerca geografica utilizzano **publicLatitude** e **publicLongitude**, non **latitude** e **longitude**. In questo modo il sistema mantiene

una relazione spaziale sufficiente per cercare libri vicini, riducendo al tempo stesso la granularità del dato esposto.

La ricerca per prossimità si basa su un'origine geografica e su un raggio selezionabile. L'origine può derivare da un indirizzo cercato dall'utente oppure dalla posizione pubblica approssimata di un libro. Il sistema considera soltanto libri pubblici, disponibili e dotati di coordinate pubbliche. Per i dati persistiti, la selezione entro il raggio e l'ordinamento per distanza sono eseguiti tramite funzioni spaziali di **PostGIS**, in particolare **ST_DWithin** e **ST_Distance**. La scelta di delegare questi calcoli al database consente di mantenere coerente il filtro geografico con la persistenza e di ordinare i risultati in base alla distanza approssimata.

Il compromesso progettuale è esplicito: ridurre la precisione della posizione può diminuire l'accuratezza del punto mostrato sulla mappa, ma protegge meglio il proprietario del libro. Per il dominio applicativo di *Culturando*, tale perdita di precisione è accettabile, perché l'obiettivo della mappa non è guidare l'utente fino a un indirizzo esatto, ma permettere di capire se una risorsa si trova in un'area ragionevolmente vicina.

4.5 Servizi esterni e strategie di fallback

Alcune funzionalità del prototipo dipendono da servizi esterni o da configurazioni opzionali. Per evitare che tali dipendenze rendano fragile l'applicazione in ambiente locale, la progettazione adotta adapter e fallback. Il geocoding utilizza Geoapify quando la variabile **GEOAPIFY_API_KEY** è configurata; in caso contrario, o in caso di fallimento, viene utilizzato Nominatim/OpenStreetMap come soluzione di riserva. L'applicazione non importa direttamente il provider nelle feature: la logica è incapsulata nel pacchetto **packages/geo**.

La catalogazione assistita utilizza il pacchetto **packages/ai**. Il recupero dei metadati bibliografici è basato su Open Library quando l'utente fornisce un ISBN, mentre l'OCR è opzionale e può essere collegato a un endpoint Cloudflare configurato tramite variabili d'ambiente [16, 17]. Il principio progettuale è che i dati suggeriti non vengano salvati automaticamente: l'utente deve poterli verificare e applicare al form.

La gestione delle immagini segue la stessa logica. Se Cloudflare R2 è configurato, copertine e thumbnail vengono salvate nello storage remoto; in caso contrario, il sistema utilizza il filesystem locale dell'applicazione web. La generazione delle thumbnail avviene in modo uniforme tramite **Sharp**, indipendentemente dalla destinazione finale

del file. Anche l'invio delle email è configurabile: con SMTP completo il sistema invia messaggi reali, mentre in sviluppo può usare la modalità console per stampare il link di verifica nei log.

Tabella 4.3: Servizi esterni e comportamento degradato

Funzionalità	Servizio principale	Fallback o comportamento locale
Geocoding	Geoapify	Nominatim/OpenStreetMap.
Metadati libro	Open Library	Nessun dato automatico; compilazione manuale.
OCR	Endpoint Cloudflare configurabile	Mock o assenza di riconoscimento automatico.
Immagini	Cloudflare R2	Salvataggio locale nel filesystem della web app.
Email	SMTP	Provider console in ambiente di sviluppo.

Questa impostazione consente al prototipo di rimanere eseguibile localmente e, al tempo stesso, di integrare servizi più robusti quando disponibili. La progettazione dei fallback non elimina i limiti dei provider esterni, ma impedisce che essi diventino parte inseparabile del dominio applicativo.

Capitolo 5

Implementazione

5.1 Organizzazione del codice applicativo

L'implementazione di *Culturando* traduce l'architettura descritta nel Capitolo 4 in una struttura di codice organizzata per responsabilità. L'applicazione web principale è sviluppata in `apps/web` mediante `Next.js` App Router: la directory delle route contiene pagine, layout e punti di ingresso HTTP, mentre la logica applicativa è demandata alle feature e ai pacchetti condivisi. Questa scelta evita che le pagine diventino contenitori di validazione, accesso ai dati o regole di dominio, mantenendo il routing come livello di composizione.

Le funzionalità specifiche della web app sono collocate in `apps/web/src/features`. Ogni feature raggruppa componenti, server action, schemi di validazione e tipi relativi a un'area del dominio. La feature di autenticazione contiene i form e le azioni di login e registrazione; la feature dei libri gestisce catalogo, dettaglio, form di creazione e modifica; la feature delle richieste implementa l'interazione tra utenti; le feature di dashboard, amministrazione, profilo e impostazioni coprono le aree protette. Questa organizzazione rende esplicito il confine tra interfaccia, validazione e operazioni persistenti, riducendo le dipendenze trasversali non necessarie.

I pacchetti in `packages` raccolgono invece funzionalità non legate a una singola pagina. Il pacchetto `packages/db` espone il client `Prisma` e lo schema del database; `packages/geo` isola geocoding, normalizzazione e approssimazione delle coordinate; `packages/ai` contiene le funzioni per catalogazione assistita, estrazione ISBN e OCR; `packages/translation` centralizza i dizionari testuali; `packages/assets` centralizza i percorsi degli asset pubblici. In questo modo le feature della web app consumano interfacce condivise senza duplicare logica tecnica.

Dal punto di vista esecutivo, le operazioni che modificano lo stato sono implementate principalmente tramite server action. Tale scelta consente di validare i dati lato server, accedere alla sessione autenticata e interagire con il database senza esporre direttamente la logica persistente al client. I form React rimangono responsabili dell'interazione utente, della raccolta dei dati e della presentazione degli errori; le server action applicano invece controlli di autorizzazione, validazione e persistenza. La separazione risponde ai requisiti RNF-04 e RNF-05, perché ogni operazione sensibile viene rieseguita sul server indipendentemente dallo stato dell'interfaccia.

5.2 Autenticazione, conferma email e profilo utente

L'autenticazione è implementata con `Auth.js` tramite provider Credentials e strategia di sessione JWT. La scelta del provider a credenziali è coerente con il modello dati del prototipo, nel quale gli utenti sono persistiti nel database applicativo e il login deve verificare esplicitamente password, stato di conferma email e ruolo. La configurazione server-side arricchisce la sessione con identificativo utente, ruolo, nickname, avatar e preferenza di saluto, permettendo alle aree protette di usare informazioni essenziali senza interrogare ripetutamente tutto il profilo.

Il flusso di registrazione parte da un form validato con `Zod`. I dati raccolti includono nome, email, password e conferma password. Prima della persistenza, l'email viene normalizzata e controllata rispetto agli account esistenti; la password non viene salvata in chiaro, ma trasformata in hash tramite una funzione server-side. L'utente creato non è immediatamente abilitato al login: il campo `emailVerifiedAt` resta nullo fino al completamento della conferma email.

La conferma dell'indirizzo è gestita attraverso token temporanei. In fase di registrazione viene generato un valore casuale, ma nel database viene salvato soltanto il relativo hash, associato all'utente e a una scadenza. Il link inviato all'utente contiene il token originale; quando il link viene aperto, il server ricalcola l'hash, verifica l'esistenza del token, controlla la scadenza e, in caso positivo, aggiorna `emailVerifiedAt`. La procedura elimina inoltre i token residui dell'utente, riducendo la permanenza di credenziali temporanee non più utili.

Il login verifica tre condizioni: presenza dell'utente, password valida e account confermato. Se la password hash non è disponibile o l'email non è stata verificata, l'autorizzazione fallisce. Questa impostazione collega l'autenticazione ai requisiti RF-01, RF-02 e RF-03, e rafforza RNF-03 perché impedisce l'accesso di account creati ma

non attivati. Quando l'account non esiste, non ha una password o non è confermato, il server esegue comunque una verifica **script** contro un hash fittizio calcolato una sola volta all'avvio, così che il tempo di risposta non distingua questi casi da una password errata e non consenta di enumerare gli account registrati. Le route operative, come dashboard, profilo, creazione libri, ricerca geografica protetta e richieste, recuperano la sessione lato server e reindirizzano o restituiscono errore quando l'identità non è presente.

La gestione del profilo completa il flusso account permettendo all'utente autenticato di modificare dati descrittivi, nickname, avatar, domicilio e visibilità pubblica. Il domicilio è trattato come informazione descrittiva del profilo e viene gestito tramite autocomplete geografico condiviso, mentre l'avatar può essere salvato su storage remoto compatibile con S3 o, in assenza di configurazione, nel filesystem locale dell'applicazione. La preferenza di saluto è mantenuta come preferenza linguistica dell'interfaccia, non come dato sensibile: viene usata per personalizzare il titolo della dashboard senza introdurre ulteriori inferenze sul profilo dell'utente.

Le operazioni più esposte ad abusi automatizzati sono protette da un limite al numero di tentativi per intervallo di tempo. Il login è limitato sia per indirizzo IP, contro il tentativo di molte credenziali da una stessa origine, sia per account, contro tentativi distribuiti su molti indirizzi verso lo stesso utente; il contatore per account viene azzerato dopo un accesso riuscito, così che un terzo possa bloccare un account solo per un intervallo breve. Registrazione e verifica di disponibilità dell'email sono limitate per indirizzo IP, poiché ogni registrazione invia un messaggio reale e la verifica rivela se un indirizzo è già registrato; la creazione di richieste è limitata per utente autenticato. Poiché l'applicazione è eseguita in ambiente serverless, dove lo stato in memoria non persiste tra le invocazioni, i contatori sono conservati in una tabella PostgreSQL con finestre temporali fisse e incremento atomico, senza introdurre un servizio esterno. Al superamento del limite l'utente riceve un messaggio localizzato che invita a riprovare più tardi; la verifica dell'email risponde invece con esito indeterminato senza bloccare la registrazione.

Queste misure riducono l'efficacia di tentativi automatizzati, ma hanno limiti noti. L'indirizzo del client è ricavato dagli header impostati dalla piattaforma di hosting ed è affidabile solo dietro un proxy fidato; la finestra fissa consente brevi picchi a cavallo tra due intervalli; un attacco distribuito su molti indirizzi e molti account resta contenuto soltanto in parte. Anche la parità dei tempi di risposta riguarda il costo dominante della verifica della password, non garantisce un tempo costante in senso stretto.

5.3 Catalogazione libraria e gestione delle immagini

La catalogazione libraria è implementata come flusso protetto: soltanto un utente autenticato può creare, modificare o cancellare libri persistiti. Il form raccoglie metadati bibliografici, stato di disponibilità, visibilità, condizione fisica, indirizzo e immagini. Anche in questo caso la validazione è eseguita lato server, così da non affidare l'integrità dei dati alla sola interfaccia client.

La creazione di un libro avviene tramite una server action che recupera la sessione, valida i dati, verifica l'eventuale duplicazione dell'ISBN all'interno della collezione dello stesso proprietario e invoca il livello repository. La persistenza crea una riga `Book`, una posizione associata `BookLocation` e zero o più immagini `BookImage`. L'associazione con il proprietario deriva dall'identificativo della sessione e non da un campo modificabile del form; in questo modo il client non può dichiarare arbitrariamente la proprietà del libro.

Le operazioni di modifica ed eliminazione applicano lo stesso principio di ownership. Prima di aggiornare o cancellare una scheda, il sistema recupera il proprietario persistito e lo confronta con l'utente autenticato. Se i valori non coincidono, l'operazione viene bloccata o rediretta. Questa verifica risponde al requisito RF-07 e al requisito non funzionale RNF-04, perché impedisce a un utente di intervenire su risorse altrui anche nel caso in cui conosca l'identificativo della route.

La gestione delle immagini supporta tre casi: caricamento diretto, URL indicati manualmente e copertine provenienti da Open Library. Per i file caricati, il sistema accetta soltanto immagini JPG, PNG o WebP entro una dimensione massima, genera nomi non prevedibili e produce thumbnail WebP tramite `Sharp`. Se Cloudflare R2 è configurato, immagine e thumbnail vengono salvate nello storage remoto; in caso contrario vengono scritte sotto gli upload pubblici locali della web app. La prima immagine salvata è marcata come principale, mentre le altre restano associate alla scheda come immagini secondarie.

Lo stato di disponibilità e la visibilità incidono direttamente sul comportamento pubblico del catalogo. I libri possono essere pubblici o privati; possono inoltre essere disponibili, consultabili, prestabili o non disponibili. Il catalogo pubblico e la ricerca geografica considerano tali attributi per evitare che una risorsa privata o indisponibile venga proposta come disponibilità territoriale. Il dettaglio del libro registra inoltre le visualizzazioni attraverso `BookStats`: l'apertura della pagina aggiorna o crea il contatore persistente, che viene poi riutilizzato nella dashboard personale e nell'area

amministrativa.

5.4 Catalogazione assistita tramite ISBN, OCR e metadati esterni

La catalogazione assistita ha lo scopo di ridurre il lavoro manuale dell'utente senza sostituirne il controllo. L'implementazione è collocata nel pacchetto `packages/ai` e viene richiamata dalla web app tramite server action dedicate. Il pacchetto fornisce tre capacità principali: estrazione e validazione di ISBN da testo, recupero di metadati bibliografici da Open Library e invio opzionale di immagini a un endpoint OCR esterno.

Il recupero tramite ISBN normalizza il valore inserito dall'utente, interroga Open Library e costruisce una proposta contenente titolo, autori, editore, anno, lingua, categorie, descrizione e copertina quando disponibili. Per alcuni dati, come gli autori, il sistema deve eseguire richieste aggiuntive perché l'API restituisce riferimenti separati. Il risultato non viene salvato direttamente nel database: viene restituito al form come suggerimento, così che l'utente possa applicarlo esplicitamente ai campi della scheda.

Il flusso OCR è opzionale e dipende dalla configurazione di un endpoint Cloudflare. L'utente può caricare una o due immagini, ad esempio copertina e retro; la server action controlla tipo e dimensione dei file, poi invia il contenuto all'adapter OCR. La risposta può contenere testo libero e, se disponibile, metadati strutturati. Il sistema tenta di estrarre un ISBN dal testo riconosciuto; se l'ISBN è presente, lo usa per interrogare Open Library, altrimenti prova a inferire un titolo e a cercare per titolo. In ambiente locale è previsto anche un testo mock tramite variabile d'ambiente, utile per validare il flusso senza dipendere dal servizio esterno.

L'aspetto centrale è il controllo umano. L'OCR può introdurre errori di riconoscimento e le API bibliografiche possono restituire dati incompleti o non corrispondenti alla copia posseduta. Per questo motivo l'assistenza propone metadati, ma il salvataggio finale resta vincolato al form e alla conferma dell'utente. La funzionalità soddisfa RF-08 senza trasformare la catalogazione assistita in un processo automatico non verificabile.

5.5 Geolocalizzazione, query spaziali e visualizzazione cartografica

La geolocalizzazione è implementata attraverso una catena composta da inserimento dell'indirizzo, geocoding, approssimazione delle coordinate, persistenza e interrogazione spaziale. L'utente non inserisce manualmente latitudine e longitudine: compila un indirizzo, eventualmente assistito da autocomplete, e il server invoca il pacchetto `packages/geo`. Il provider principale è Geoapify quando la chiave è configurata; in caso contrario, o in caso di fallimento, il sistema prova Nominatim/OpenStreetMap come fallback. Se nessun provider restituisce coordinate, il libro può comunque essere salvato con l'indirizzo descrittivo, ma non partecipa alle ricerche territoriali basate su coordinate.

Il pacchetto geografico normalizza le coordinate e genera la versione pubblica approssimata arrotondando latitudine e longitudine a tre decimali. Le due rappresentazioni vengono salvate separatamente in `BookLocation`: `latitude` e `longitude` rappresentano il dato preciso interno, mentre `publicLatitude` e `publicLongitude` sono usati per mappa e discovery. Questa distinzione implementa la strategia di minimizzazione discussa nel Capitolo 4 e consente di soddisfare RNF-01 e RNF-02.

La ricerca per prossimità sui libri persistiti è eseguita tramite query SQL geospaziali invocate da `Prisma`. Poiché gli operatori spaziali necessari non sono esposti direttamente dal modello `Prisma` in modo sufficiente per il caso d'uso, il repository usa SQL grezzo parametrizzato con funzioni `PostGIS`. La query costruisce punti geografici a partire dalle coordinate pubbliche, filtra i risultati entro il raggio richiesto con `ST_DWithin`, calcola la distanza con `ST_Distance`, esclude il libro di origine quando necessario, considera soltanto libri pubblici e disponibili, e ordina i risultati per distanza crescente.

Il prototipo mantiene anche un fallback sui dati dimostrativi non persistiti, per i quali la distanza viene calcolata in `TypeScript` tramite formula haversine. Questo fallback non sostituisce il comportamento principale basato su database, ma permette di mantenere coerente l'esperienza anche in presenza di dati mock. Nel caso dei dati reali, la responsabilità del filtro spaziale resta invece nel database, scelta più adatta a un'evoluzione con dataset più ampi.

La visualizzazione cartografica è realizzata con `MapLibre GL JS`. La mappa mostra il punto di ricerca o il libro di origine e i libri disponibili nelle vicinanze usando le sole coordinate pubbliche. I risultati sono rappresentati tramite sorgente `GeoJSON`,

marker, cluster e popup con collegamento al dettaglio della scheda. L'interfaccia include controlli per vista 2D/3D e gestione della camera; sui viewport compatti riduce i comportamenti più pesanti, come rotazione automatica e impostazioni tridimensionali iniziali, e rispetta `prefers-reduced-motion`. In questo modo la componente cartografica supporta RF-11 e RF-12 senza sacrificare l'usabilità mobile prevista da RNF-10.

5.6 Richieste tra utenti, dashboard e amministrazione

Il flusso delle richieste trasforma la discovery di un libro in un'interazione gestibile tra utenti. Nel dettaglio del libro, un utente autenticato può inviare una richiesta di consultazione, prestito o informazione. La server action recupera la sessione, verifica che il libro esista come risorsa pubblica e richiedibile, controlla che il richiedente non coincida con il proprietario e valida tipo e messaggio della richiesta. Solo dopo questi controlli viene creata una riga `LoanRequest` con stato iniziale `pending`.

La gestione delle richieste è divisa in due prospettive. Il proprietario visualizza le richieste ricevute nella dashboard e può accettare o rifiutare solo quelle ancora in attesa. L'aggiornamento dello stato richiede che l'utente autenticato corrisponda al proprietario persistito della richiesta. Il richiedente, invece, dispone di una vista sulle richieste inviate e può cancellare quelle ancora `pending`. Anche questa operazione è vincolata all'identificativo del richiedente, così da impedire modifiche su richieste di altri utenti.

La dashboard personale aggrega informazioni operative utili all'utente: numero di libri pubblicati, distinzione tra libri pubblici e privati, visualizzazioni totali, richieste ricevute, richieste pendenti, richieste accettate e libri più visualizzati. Questi dati sono calcolati tramite query e aggregazioni sul database, non tramite stato client. La dashboard non ha quindi solo funzione di navigazione, ma fornisce una sintesi dell'attività collegata alla collezione dell'utente.

L'area amministrativa è accessibile soltanto a utenti con ruolo `admin` nella sessione. Essa presenta metriche aggregate del prototipo, tra cui numero di utenti, libri, libri pubblici e privati, richieste per stato, visualizzazioni totali, ultimi utenti registrati e libri pubblicati più recentemente. Nel contesto della tesi, questa funzionalità non viene presentata come strumento gestionale completo per una piattaforma in

produzione, ma come supporto alla verifica dello stato del prototipo e all'osservabilità funzionale prevista da RNF-12.

Nel complesso, il Capitolo 5 mostra come i requisiti definiti nel Capitolo 3 siano stati realizzati attraverso una combinazione di route protette, server action, validazione, repository persistenti, pacchetti condivisi e controlli di autorizzazione. La sezione successiva dell'elaborato può quindi concentrarsi sulla validazione: non più sulla descrizione delle componenti, ma sulla verifica del loro comportamento rispetto ai requisiti funzionali e non funzionali.

Capitolo 6

Testing e validazione

6.1 Strategia di verifica

6.1.1 Obiettivi della validazione

La validazione di *Culturando* ha l'obiettivo di verificare che il prototipo realizzato sia coerente con i requisiti definiti nel Capitolo 3 e con le scelte implementative descritte nel Capitolo 5. In particolare, la verifica non riguarda soltanto la capacità dell'applicazione di essere compilata, ma anche la corrispondenza tra requisiti funzionali, comportamenti osservabili e vincoli non funzionali, con attenzione specifica ad autenticazione, persistenza, geolocalizzazione e tutela della posizione.

Il processo di validazione è stato organizzato su due livelli complementari. Il primo livello riguarda la qualità tecnica del codice: type checking, linting, controllo di formattazione, test unitari automatizzati, test di integrazione sui vincoli di autorizzazione e build di produzione. Questi controlli permettono di individuare errori statici, violazioni di convenzioni e problemi di integrazione prima dell'esecuzione manuale degli scenari utente. Il secondo livello riguarda la validazione funzionale del prototipo: registrazione, conferma email, login, gestione dei libri, catalogazione assistita, ricerca, richieste, dashboard e comportamento geografico.

La relazione adottata è di tipo tracciabile: ogni requisito rilevante viene collegato a uno o più casi d'uso, a un controllo tecnico o funzionale e a un esito verificabile. Tale impostazione evita di presentare la validazione come una lista generica di prove e consente invece di mostrare in che modo le funzionalità implementate rispondano agli obiettivi stabiliti in fase di analisi.

Nel contesto del presente elaborato, la validazione non viene presentata come

una certificazione produttiva del sistema. Il progetto resta un prototipo universitario: dispone di una suite di test unitari automatizzati limitata alla logica pura e di un insieme circoscritto di test di integrazione dedicati ai vincoli di proprietà sulle operazioni relative a libri e richieste, oltre che di un unico smoke test end-to-end sul percorso critico, dalla registrazione all'accettazione di una richiesta; manca invece una suite end-to-end estesa agli altri flussi. La strategia adottata combina quindi controlli automatizzati disponibili nel repository e scenari manuali ripetibili, sufficienti a verificare i flussi principali e i vincoli progettuali più rilevanti.

6.1.2 Livelli di verifica adottati

I livelli di verifica adottati sono sintetizzati in table 6.1. Essi coprono progressivamente correttezza statica, producibilità della build, riproducibilità dell'ambiente e validazione funzionale.

Tabella 6.1: Livelli di verifica adottati

Livello	Strumenti o attività	Obiettivo
Verifica statica	TypeScript, ESLint, Biome, regole Nx	Individuare errori di tipo, violazioni sintattiche, problemi di formattazione e incoerenze rispetto alle convenzioni del workspace.
Test unitari	Vitest tramite Nx	Verificare in modo automatico e ripetibile la logica pura: approssimazione delle coordinate pubbliche, hashing delle password, schemi di validazione dei form e riconoscimento del ruolo amministrativo.
Test di integrazione	Vitest su database PostgreSQL dedicato	Verificare in modo automatico che le operazioni lato server su libri e richieste rispettino i vincoli di proprietà e di autenticazione.
Smoke test end-to-end	Playwright su build di produzione	Verificare in modo automatico, con browser e server HTTP reali, il percorso critico da registrazione e conferma email fino a pubblicazione di un libro, richiesta e accettazione.
Build e integrazione	Build Next.js tramite Nx	Verificare che l'applicazione possa essere compilata in modalità produzione e che le route principali siano riconosciute dal framework.
Ambiente locale	Script di sviluppo, database PostgreSQL e PostGIS	Verificare la riproducibilità dell'ambiente necessario al prototipo e la disponibilità dei servizi locali.
Scenari funzionali	Prove manuali sui flussi principali	Verificare requisiti osservabili relativi ad account, libri, ricerca, richieste, dashboard e geolocalizzazione.
Privacy geografica	Controlli su coordinate pubbliche e query nearby	Verificare che mappa e discovery usino coordinate approssimate e non coordinate precise.

La distinzione tra livelli è rilevante perché non tutti gli errori emergono nello stesso momento. Un progetto può superare la compilazione **TypeScript** ma fallire la build di produzione; allo stesso modo, una build corretta non dimostra automaticamente che i vincoli di proprietà, disponibilità o privacy siano rispettati durante l'interazione utente. Per questa ragione la validazione del prototipo richiede sia strumenti automatici sia scenari applicativi mirati.

Dal punto di vista operativo, verifica statica, test unitari e build sono eseguiti automaticamente da una pipeline di integrazione continua basata su GitHub Actions, attivata a ogni pull request e a ogni aggiornamento del ramo principale. La stessa pipeline applica le migration **Prisma** a un database **PostgreSQL/PostGIS** temporaneo, esegue su di esso i test di integrazione e segnala eventuali divergenze tra lo schema dichiarato e le migration versionate. I controlli automatici non dipendono quindi da un'esecuzione manuale; restano invece esclusi dalla pipeline il deploy, lo smoke test end-to-end, eseguito per ora soltanto in locale, e gli scenari funzionali manuali.

6.2 Verifica statica e qualità del codice

La verifica statica è stata eseguita mediante i comandi previsti dal repository. Gli esiti sono riportati in table 6.2. I controlli sono stati eseguiti dopo la correzione di una difformità puramente formale individuata da Biome nel file di configurazione dell'autenticazione; tale correzione non ha modificato il comportamento applicativo.

6.2.1 Type checking

Il controllo di tipo è stato eseguito con il comando di type checking dedicato alla web app:

```
1 pnpm exec tsc -p apps/web/tsconfig.json --noEmit
```

L'opzione `-noEmit` permette di verificare il progetto senza generare file di output, concentrandosi quindi sulla correttezza statica del codice **TypeScript**. Il comando si è concluso senza errori o diagnostiche, confermando che il codice della web app è coerente con la configurazione **TypeScript** attuale.

6.2.2 Linting e convenzioni

Il linting è stato eseguito con `pnpm lint`, che invoca il target `lint` dell'applicazione web tramite `Nx`. Il controllo ha completato correttamente l'esecuzione, senza segnalare errori bloccanti. Questo risultato verifica la conformità del codice alle regole ESLint configurate per il progetto e contribuisce a mantenere coerenti le convenzioni tra route, feature e componenti.

Il controllo Biome è stato eseguito con `pnpm biome:ci`. Una prima esecuzione ha evidenziato una difformità di formattazione in `apps/web/src/config/auth.ts`; dopo la correzione, il comando ha verificato 170 file senza applicare fix e senza produrre errori. L'esito conferma che, allo stato finale della validazione, il repository rispetta le regole di formattazione e controllo gestite da Biome.

6.2.3 Verifica della build

La build di produzione è stata verificata con `pnpm build`. Il comando invoca `Nx` e compila l'applicazione `Next.js`. L'esecuzione ha completato correttamente la compilazione, il controllo TypeScript interno alla build, la raccolta dei dati di pagina, la generazione delle pagine statiche e l'ottimizzazione finale. Il framework ha inoltre riconosciuto le route applicative principali, comprese le aree pubbliche, le route di autenticazione, i dettagli dei libri, le pagine dashboard e la ricerca nearby.

Durante la build è stato prodotto un avviso non bloccante relativo al supporto incompleto dei project references TypeScript da parte del processo di build `Next.js`, con tentativo di compilazione incrementale. L'avviso non ha impedito il completamento della build e non è stato classificato come errore funzionale del prototipo.

Tabella 6.2: Esiti della verifica statica, di build e dei test automatizzati

ID	Controllo	Esito	Osservazioni
VQ-01	Type checking TypeScript	Superato	Nessuna diagnostica TypeScript.
VQ-02	Linting ESLint/Nx	Superato	Target lint dell'applicazione web completato da Nx.
VQ-03	Controllo Biome CI	Superato	170 file verificati dopo correzione formale; nessun fix applicato nello stato finale.

ID	Controllo	Esito	Osservazioni
VQ-04	Build di produzione	Superato	Build <code>Next.js</code> completata; 14/14 pagine statiche generate; route applicative riconosciute.
VQ-06	Test unitari <code>Vitest</code>	Superato	178 test superati in 9 file, dopo la correzione di due difetti individuati dalla suite (Sezione 6.2.4).
VQ-07	Test di integrazione <code>Vitest</code>	Superato	31 test superati in 9 file su database <code>PostgreSQL</code> di test locale (Sezione 6.2.5).
VQ-08	Smoke test end-to-end <code>Playwright</code>	Superato	1 test superato in 4 esecuzioni locali consecutive su build di produzione e database dedicato (Sezione 6.2.6).

6.2.4 Test unitari

I test unitari sono realizzati con `Vitest`, integrato nel monorepo tramite il plugin `@nx/vitest`. Il plugin deduce un target `test` per ogni progetto che contiene un file di configurazione `Vitest`; la suite viene eseguita con `pnpm test`, che invoca il target su tutti i progetti configurati. I file di test adottano la convenzione `*.spec.ts` e sono collocati accanto al modulo verificato.

La suite si concentra sulla logica pura, cioè sulle funzioni che non dipendono da database, rete o interfaccia e che possono quindi essere verificate in modo deterministico. Sono state considerate tre aree. La prima è l'approssimazione delle coordinate nel pacchetto `packages/geo`, che implementa RNF-01 e RNF-02: i test verificano l'arrotondamento a tre decimali, la gestione di valori non validi o fuori intervallo, l'idempotenza della trasformazione e la minore precisione del valore pubblico rispetto a quello interno. La seconda è l'hashing delle password in `apps/web/src/lib/password.ts`, collegato a RNF-03: i test verificano che salt casuali producano hash diversi per la stessa password, che la verifica accetti solo la password corretta e che hash malformati

non generino eccezioni. La terza comprende gli schemi di validazione **Zod** dei form di registrazione, login, libro, profilo e richiesta, collegati a RNF-05: per ciascuno schema sono verificati input valido, campi obbligatori mancanti e vincoli specifici, come i limiti di lunghezza della password e la coincidenza tra password e conferma. Un ulteriore test verifica la funzione che riconosce una sessione con ruolo amministrativo, estratta dalla pagina della dashboard amministrativa per poter essere verificata in isolamento, e un altro l'estrazione dell'indirizzo del client dagli header del proxy, usata dalla limitazione dei tentativi.

L'esecuzione ha prodotto 178 test superati, distribuiti su 9 file e due progetti **Nx**. La scrittura della suite ha fatto emergere due difetti, inizialmente documentati come fallimenti previsti e successivamente corretti. Il primo riguardava la verifica della password: se la parte esadecimale dell'hash memorizzato non era valida, la chiave decodificata risultava vuota e il confronto accettava qualsiasi password. Il difetto non si manifestava con hash generati dall'applicazione, che hanno sempre formato corretto, ma rendeva la funzione non robusta rispetto a dati corrotti; la correzione rifiuta ogni chiave memorizzata la cui lunghezza non coincide con quella prevista dalla derivazione. Il secondo riguardava la normalizzazione delle coordinate: valori `null` venivano convertiti nel punto (0,0) anziché essere rifiutati. Il caso è discusso nella Sezione 6.4.2. Dopo le correzioni, i test corrispondenti verificano il comportamento atteso come casi ordinari.

6.2.5 Test di integrazione

I test di integrazione verificano i vincoli di autorizzazione collegati a RNF-04 nelle server action che modificano libri e richieste. A differenza dei test unitari, ogni caso invoca l'azione reale contro un database **PostgreSQL** dedicato, il cui schema è ottenuto applicando le stesse migration **Prisma** usate negli altri ambienti. Sono sostituite con doppi di test soltanto la lettura della sessione autenticata, così da simulare di volta in volta il proprietario, un altro utente o un visitatore non autenticato, e le funzioni del framework **Next.js** legate a navigazione e cache. L'esito viene verificato sia sul valore restituito dall'azione sia sullo stato effettivamente persistito.

I casi coperti riguardano la modifica e l'eliminazione di un libro, consentite al solo proprietario; la creazione di una richiesta, negata al proprietario del libro e agli utenti non autenticati; l'accettazione e il rifiuto, riservati al proprietario e limitati alle richieste in attesa; l'annullamento, riservato al richiedente e anch'esso limitato alle

richieste in attesa. La suite è eseguita con un comando distinto da quello dei test unitari, in modo sequenziale, e rifiuta di operare su un database il cui indirizzo di connessione non identifichi esplicitamente un ambiente di test, poiché ogni caso svuota le tabelle coinvolte prima dell'esecuzione. Un secondo gruppo di test riguarda la limitazione dei tentativi descritta nella Sezione 5.2. I test verificano il comportamento del limitatore sul database reale, cioè il blocco al superamento della soglia, la riapertura alla scadenza della finestra, l'azzeramento dopo un accesso riuscito e l'assenza di incrementi persi con richieste concorrenti, e il suo collegamento a login, registrazione, verifica dell'email e creazione di richieste. Un ulteriore test verifica che, per un account inesistente o non confermato, venga comunque eseguita la verifica della password contro l'hash fittizio; il test controlla l'invocazione della verifica e non misura tempi reali. L'esecuzione locale ha prodotto 31 test superati in 9 file.

La copertura resta circoscritta. Il controllo del ruolo amministrativo è verificato dal solo test unitario della funzione dedicata e non attraverso il rendering della pagina, mentre le operazioni su profilo, catalogazione, ricerca e geolocalizzazione non dispongono di test di integrazione.

6.2.6 Smoke test end-to-end

Lo smoke test end-to-end, realizzato con **Playwright**, percorre in un browser reale il flusso considerato critico per il prototipo: registrazione di un proprietario, conferma dell'indirizzo email, accesso, pubblicazione di un libro, registrazione e accesso di un secondo utente, invio di una richiesta e sua accettazione da parte del proprietario. A differenza dei test di integrazione, nessun componente dell'applicazione viene sostituito: il test interagisce con la build di produzione servita via HTTP, e i due utenti operano in contesti di navigazione distinti, ciascuno con la propria sessione `Auth.js` basata su cookie. L'esito è verificato sull'interfaccia, attraverso il passaggio della richiesta allo stato accettato.

L'ambiente di esecuzione è isolato da quello di sviluppo. Il server utilizza un database **PostgreSQL** dedicato, distinto sia da quello di sviluppo sia da quello dei test di integrazione, a cui vengono applicate le migration versionate prima dell'esecuzione. L'invio delle email è configurato in modalità di sola registrazione su log: il test ricava il collegamento di conferma dall'output del server, senza ricorrere a una casella di posta reale, poiché nel database viene memorizzata soltanto l'impronta del token. Email e titolo del libro contengono un identificativo diverso per ogni esecuzione, così che

esecuzioni ripetute non entrino in conflitto senza richiedere un ripristino del database.

La creazione del libro dipende da servizi esterni non controllati dal prototipo: i suggerimenti di indirizzo e il geocoding lato server interrogano Nominatim, mentre la ricerca della copertina interroga Open Library. Poiché il geocoding non prevede un timeout applicativo, i passaggi che attendono questi servizi hanno limiti di tempo espliciti, in modo che un rallentamento esterno produca un fallimento circoscritto e riconoscibile, distinto da un difetto dell'applicazione. Quattro esecuzioni locali consecutive sono state completate con successo.

La copertura resta deliberatamente minima. Il test verifica un solo percorso con esito positivo, su un unico browser, e non comprende casi negativi, che per le autorizzazioni sono affidati ai test di integrazione, né ricerca, mappa, catalogazione assistita e gestione del profilo. Non è inoltre ancora eseguito dalla pipeline di integrazione continua.

6.2.7 Riproducibilità dell'ambiente

La riproducibilità locale è stata verificata indirettamente attraverso l'esecuzione della build, l'uso del database PostgreSQL/PostGIS in ambiente Docker e le prove funzionali condotte sulla stessa istanza locale. Non è stata invece documentata in questa sessione una verifica completa di provisioning o deploy produttivo; per questo RNF-08 viene mantenuto come esito parziale nella matrice conclusiva.

6.3 Validazione dei requisiti funzionali

La validazione funzionale è stata impostata a partire dai casi d'uso del Capitolo 3. Ogni scenario verifica un comportamento osservabile del prototipo e, dove necessario, un vincolo di autorizzazione o integrità. A differenza dei controlli statici descritti nella sezione precedente, questi casi richiedono l'interazione con l'applicazione in esecuzione e con un database popolato con dati coerenti. Per questo motivo, gli esiti delle prove funzionali sono stati aggiornati solo dopo una sessione manuale o tecnica documentata.

I casi di test sono stati raggruppati per area funzionale: account e autenticazione, patrimonio librario, catalogazione assistita, ricerca e discovery, condivisione tramite richieste. La validazione specifica della geolocalizzazione è trattata separatamente nella Sezione 6.4, perché costituisce un vincolo trasversale sia funzionale sia di privacy.

6.3.1 Account e autenticazione

Gli scenari relativi ad account e autenticazione verificano che il sistema consenta la creazione di un utente, protegga l'accesso fino alla conferma email e rifiuti casi non validi. Le prove sono collegate ai requisiti RF-01, RF-02, RF-03, RNF-03 e RNF-04, perché coinvolgono registrazione, verifica dell'indirizzo, login e protezione delle route operative.

Tabella 6.3: Casi di test per account e autenticazione

ID	Scenario	Risultato atteso	Esito
TC-AUTH-01	Registrazione con dati validi.	L'utente viene creato non verificato, il token di conferma viene generato e il sistema mostra il messaggio di invio email.	Superato
TC-AUTH-02	Registrazione con email già presente.	La registrazione viene rifiutata e viene mostrato un errore sul campo email.	Superato
TC-AUTH-03	Apertura di un link di conferma con token valido.	L'account viene marcato come verificato, i token residui vengono eliminati e l'utente può procedere al login.	Superato
TC-AUTH-04	Apertura di un link con token mancante, invalido o scaduto.	La conferma viene rifiutata e l'utente visualizza uno stato di errore senza attivazione dell'account.	Superato

Un ulteriore controllo implicito del flusso riguarda l'accesso alle aree protette. Dashboard, profilo, impostazioni, creazione libri, richieste e ricerca nearby richiedono una sessione valida. La mancata autenticazione deve quindi produrre redirect o messaggi di errore coerenti, senza eseguire operazioni persistenti.

La prima sessione manuale locale ha confermato il funzionamento del percorso positivo di registrazione e verifica email. Dopo la configurazione SMTP del provider transazionale, il sistema ha inviato correttamente il messaggio di conferma; l'apertura del collegamento ricevuto ha aggiornato l'utente come verificato e ha rimosso il token residuo dal database. Il successivo accesso con l'utenza verificata ha confermato che il

blocco sul login viene rimosso dopo la conferma dell'indirizzo. La stessa sessione ha verificato il rifiuto di una registrazione con email già presente. Per verificare il caso di scadenza del token è stata eseguita una prova locale riducendo temporaneamente la durata del collegamento di conferma: dopo la scadenza, la pagina di verifica ha rifiutato il token e ha mostrato un messaggio che richiede una nuova registrazione.

La sessione manuale ha inoltre verificato alcuni flussi successivi all'autenticazione. L'utente confermato accede correttamente alla dashboard personale, può modificare i dati del profilo, aggiornare l'avatar e salvare le impostazioni applicative; le modifiche risultano persistenti dopo la navigazione successiva. La preferenza di saluto non è stata considerata come scenario autonomo di validazione perché non è più esposta come scelta esplicita nel flusso utente corrente. L'accesso alla dashboard amministrativa è stato verificato con un account locale dotato del ruolo `admin`, confermando la raggiungibilità dell'area riservata agli amministratori.

6.3.2 Gestione del patrimonio librario

La validazione della gestione libraria riguarda la corretta persistenza dei libri e il rispetto della proprietà. Gli scenari verificano RF-05, RF-06, RF-07, RF-09 e, indirettamente, RNF-04 e RNF-05. La creazione e la modifica devono salvare metadati, immagini e posizione descrittiva; le operazioni di aggiornamento ed eliminazione devono essere consentite solo al proprietario.

Tabella 6.4: Casi di test per la gestione del patrimonio librario

ID	Scenario	Risultato atteso	Esito
TC-BOOK-01	Creazione di una scheda libro con dati validi.	Il libro viene associato all'utente autenticato, salvato nel database e reso visibile secondo la visibilità scelta.	Superato
TC-BOOK-02	Modifica di un libro posseduto dall'utente.	I campi modificati vengono aggiornati e la pagina di dettaglio mostra i nuovi dati.	Superato
TC-BOOK-03	Eliminazione di un libro posseduto dall'utente.	La scheda viene rimossa e l'utente viene riportato alla dashboard o al catalogo senza riferimenti residui.	Superato

ID	Scenario	Risultato atteso	Esito
TC-BOOK-04	Pubblicazione o modifica di un libro con visibilità privata.	Il libro resta gestibile dal proprietario ma non viene proposto nel catalogo pubblico o nelle ricerche territoriali pubbliche.	Superato
TC-BOOK-05	Inserimento di un ISBN già presente nella collezione dello stesso proprietario.	Il salvataggio viene bloccato e il sistema segnala la duplicazione dell'ISBN.	Superato

Per le immagini, la prova funzionale ha considerato sia il caricamento diretto sia la presenza di thumbnail. Un file valido deve produrre un'immagine associata al libro e, quando il processamento riesce, una versione ridotta utilizzabile nelle card e nel dettaglio. La sessione manuale ha confermato il corretto completamento dell'intero procedimento: caricamento dell'immagine, associazione alla scheda libro, persistenza dopo il salvataggio e visualizzazione della copertina nelle viste applicative previste.

La sessione manuale sulla gestione libraria ha confermato i flussi principali del proprietario: un utente autenticato può creare una scheda libro compilando le informazioni minime richieste, visualizzarla nella dashboard personale, modificarne i dati e cancellarla correttamente. La prova sulla visibilità ha inoltre confermato che un libro privato resta gestibile dal proprietario ma non viene incluso nella vista pubblica, nella quale il filtro mostra soltanto inserzioni pubbliche. Il controllo di unicità dell'ISBN nella collezione dello stesso proprietario è stato verificato tentando l'inserimento di un secondo libro con identificativo già presente: il sistema ha bloccato il salvataggio segnalando la duplicazione.

6.3.3 Catalogazione assistita

La catalogazione assistita verifica RF-08 e parte di RNF-05. Il comportamento atteso non è la persistenza automatica dei dati recuperati, ma la generazione di suggerimenti bibliografici che l'utente può controllare prima del salvataggio. Gli scenari considerano quindi sia il recupero tramite ISBN, sia l'uso opzionale dell'OCR, sia la gestione dei casi in cui il dato non sia presente o non sia valido.

Tabella 6.5: Casi di test per catalogazione assistita

ID	Scenario	Risultato atteso	Esito
TC-CAT-01	Lookup di un ISBN valido tramite Open Library.	Il sistema normalizza l'ISBN e restituisce una proposta con metadati bibliografici disponibili.	Superato
TC-CAT-02	Applicazione dei metadati suggeriti al form libro.	Titolo, autore, editore, anno, lingua, categoria, descrizione, ISBN e copertina vengono applicati come campi modificabili, senza salvataggio automatico.	Superato
TC-CAT-03	ISBN mancante, inesistente o formalmente invalido.	La procedura restituisce un esito controllato e non produce dati persistenti.	Superato
TC-CAT-04	OCR configurato o simulato tramite testo mock.	Il testo riconosciuto viene analizzato, l'ISBN viene estratto quando valido e i metadati OCR vengono combinati con l'eventuale lookup bibliografico.	Superato
TC-CAT-05	Immagine OCR assente, troppo grande, non supportata o errore del provider.	L'operazione viene rifiutata con uno stato d'errore specifico e il form resta compilabile manualmente.	Superato

La validazione tecnica ha confermato l'estrazione di ISBN-13 e ISBN-10 con controllo del checksum e lo scarto di un identificativo formalmente invalido. Il lookup Open Library eseguito su 9780140328721 ha restituito una proposta per *Fantastic Mr. Fox*, includendo autore, editore, anno, lingua e copertina. L'adapter OCR è stato verificato simulando una risposta positiva con testo e metadati e una risposta HTTP fallita: nel primo caso il risultato è stato normalizzato come riconoscimento riuscito, nel secondo è stato restituito un errore controllato. L'ispezione dell'integrazione nel form conferma infine che i suggerimenti vengono applicati ai campi dell'interfaccia e

che la persistenza avviene solo dopo la successiva conferma dell'utente.

6.3.4 Ricerca e discovery

La ricerca bibliografica verifica che il catalogo pubblico consenta di individuare libri tramite metadati descrittivi, mentre l'accesso alla ricerca geografica verifica la distinzione tra consultazione pubblica e funzionalità protetta. Gli scenari sono collegati a RF-09, RF-10, RF-11 e RF-12.

Tabella 6.6: Casi di test per ricerca e discovery

ID	Scenario	Risultato atteso	Esito
TC- SEARCH- 01	Ricerca nel catalogo per titolo, autore, ISBN, editore, città, categoria o descrizione.	Il catalogo filtra i risultati coerentemente con la query e mantiene visibili solo le schede ammesse alla consultazione pubblica.	Superato
TC- SEARCH- 02	Accesso alla ricerca nearby senza sessione autenticata.	L'utente non autenticato viene indirizzato al login e non accede alla funzionalità geografica protetta.	Superato

La separazione tra catalogo e nearby è rilevante dal punto di vista dei requisiti. Il catalogo rappresenta una vetrina pubblica del patrimonio condiviso; la ricerca geografica, invece, usa dati di posizione e viene quindi mantenuta dietro autenticazione, coerentemente con la maggiore sensibilità del dato trattato. La sessione manuale sul catalogo ha verificato la visibilità delle inserzioni pubbliche e l'esclusione dei libri privati dai risultati. La ricerca testuale è stata provata con query su titolo, autore, ISBN, città, categoria e descrizione, ottenendo filtri coerenti con i dati inseriti. Inoltre, l'accesso alla funzionalità nearby senza sessione autenticata è risultato correttamente bloccato, confermando la separazione tra consultazione pubblica del catalogo e funzioni geografiche protette.

6.3.5 Condivisione e richieste

La validazione delle richieste verifica che l'interazione tra utenti avvenga solo quando il libro è pubblico, richiedibile e appartenente a un altro utente. Gli scenari sono

collegati a RF-13, RF-14, RF-15 e RNF-04. Il comportamento atteso non riguarda soltanto la creazione del record, ma anche il controllo degli stati e delle autorizzazioni.

Tabella 6.7: Casi di test per condivisione e richieste

ID	Scenario	Risultato atteso	Esito
TC-REQ-01	Invio di una richiesta valida per un libro pubblico di un altro utente.	La richiesta viene salvata con stato pending e compare tra le richieste ricevute dal proprietario.	Superato
TC-REQ-02	Tentativo di richiesta verso un libro posseduto dallo stesso utente.	Il sistema blocca l'operazione e segnala che non è possibile richiedere un proprio libro.	Superato
TC-REQ-03	Tentativo di richiesta senza autenticazione.	La richiesta non viene creata e l'utente riceve indicazione di autenticarsi.	Superato
TC-REQ-04	Richiesta verso libro privato, indisponibile o non esistente.	L'operazione viene rifiutata e non vengono creati record persistenti.	Superato
TC-REQ-05	Accettazione o rifiuto di una richiesta ricevuta dal proprietario.	Solo il proprietario può modificare una richiesta pending ; lo stato viene aggiornato in accepted o rejected .	Superato

La validazione deve inoltre considerare la prospettiva del richiedente: una richiesta inviata e ancora in stato **pending** deve poter essere cancellata dal suo autore, mentre richieste non appartenenti all'utente o già gestite non devono essere modificabili arbitrariamente. La sessione manuale ha confermato il flusso completo tra due utenti distinti: invio di una richiesta verso un libro pubblico di altro proprietario, visualizzazione della richiesta tra quelle ricevute, blocco della richiesta verso un proprio libro, gestione dello scenario non autenticato, impossibilità di richiedere libri privati, accettazione e rifiuto da parte del proprietario e cancellazione corretta di una richiesta ancora pendente dal lato del richiedente. Questo controllo completa il requisito di autorizzazione sulle interazioni tra utenti.

6.4 Validazione della geolocalizzazione e della privacy

La geolocalizzazione rappresenta l'area di validazione più delicata del prototipo, perché combina utilità funzionale e tutela di un dato potenzialmente sensibile. Le prove descritte in questa sezione verificano che il sistema sia in grado di trasformare un indirizzo in coordinate, salvare una rappresentazione pubblica approssimata e usare tale rappresentazione nelle ricerche e nella mappa. L'obiettivo non è misurare sperimentalmente la precisione dei provider di geocoding, ma verificare che il comportamento applicativo rispetti il modello progettuale definito nei capitoli precedenti.

La validazione geografica è distinta in quattro gruppi: geocoding e fallback, approssimazione delle coordinate, ricerca per prossimità e visualizzazione cartografica. I primi due gruppi verificano la produzione e la conservazione del dato geografico; gli ultimi due verificano il suo utilizzo operativo. La distinzione è importante perché la tutela della privacy non dipende soltanto da come il dato viene salvato, ma anche da quali campi vengono usati nelle query e nell'interfaccia.

6.4.1 Geocoding e fallback

Il geocoding viene validato verificando il comportamento del pacchetto geografico quando è disponibile il provider principale e quando invece è necessario ricorrere al provider di riserva. Nel prototipo, Geoapify rappresenta il provider principale configurabile tramite variabile d'ambiente, mentre Nominatim/OpenStreetMap funge da fallback. La validazione deve considerare anche il caso in cui nessun provider restituisca coordinate: in tale situazione il libro può conservare l'indirizzo descrittivo, ma non deve essere incluso in ricerche territoriali che richiedono coordinate pubbliche.

Tabella 6.8: Casi di test per geocoding e fallback

ID	Scenario	Risultato atteso	Esito
TC-GEO-01	Geocoding con Geoapify configurato e indirizzo valido.	Il sistema tenta il provider principale, normalizza le coordinate ottenute e genera le coordinate pubbliche approssimate.	Parziale: chiama live non rieseguita

ID	Scenario	Risultato atteso	Esito
TC-GEO-02	Geocoding senza Geopify configurato o con provider principale non disponibile.	Il sistema tenta il fallback Nominatim/OpenStreetMap e, se ottiene un risultato valido, produce la stessa struttura geografica attesa.	Superato
TC-GEO-03	Geocoding senza risultati validi.	Il libro mantiene l'indirizzo testuale, ma non dispone di coordinate pubbliche e non partecipa alle ricerche nearby.	Superato

Questi test non assumono che un provider esterno restituisca sempre lo stesso punto per lo stesso indirizzo, perché tale comportamento dipende da basi dati e politiche del provider. L'elemento da verificare per la tesi è la robustezza del flusso applicativo: normalizzazione del risultato, fallback controllato e gestione dell'assenza di coordinate.

La sessione di validazione ha confermato a livello tecnico che la funzione di geocoding produce un risultato nullo quando non sono disponibili coordinate valide e che il flusso di creazione del libro salva i campi geografici in modo opzionale. La verifica live del provider principale non è stata ripetuta nella sessione locale, mentre il ramo di fallback e la gestione dell'assenza di coordinate risultano coperti dall'ispezione del codice e dai controlli sulle query territoriali.

6.4.2 Approssimazione delle coordinate

La prova sulle coordinate verifica direttamente RNF-01 e RNF-02. Il sistema deve distinguere tra coordinate precise e coordinate pubbliche: le prime rappresentano il dato interno, le seconde sono quelle utilizzabili per discovery e visualizzazione. Nel codice corrente, l'approssimazione avviene arrotondando latitudine e longitudine a tre cifre decimali.

Tabella 6.9: Casi di test per approssimazione e privacy delle coordinate

ID	Scenario	Risultato atteso	Esito
TC-PRIV-01	Trasformazione di coordinate precise valide.	Le coordinate pubbliche vengono generate arrotondando latitudine e longitudine a tre decimali.	Superato
TC-PRIV-02	Consultazione dei dati usati da nearby e mappa.	Query spaziali e marker utilizzano <code>publicLatitude</code> e <code>publicLongitude</code> , non i campi precisi.	Superato
TC-PRIV-03	Libro privo di coordinate pubbliche.	Il libro non viene incluso tra i risultati geografici e non produce marker sulla mappa nearby.	Superato

Il criterio di successo non consiste nell’ottenere una posizione anonima in senso assoluto, risultato che richiederebbe tecniche e valutazioni più avanzate, ma nel dimostrare che il prototipo non espone il punto preciso associato alla scheda libro nelle funzionalità pubbliche o semi-pubbliche di discovery. La limitazione della granularità geografica è quindi una misura progettuale di minimizzazione, coerente con la natura prototipale del sistema.

La prova runtime dell’approssimazione ha verificato che coordinate precise come 40.851234 e 14.268789 vengono trasformate rispettivamente in 40.851 e 14.269. L’ispezione delle query e dei componenti cartografici conferma inoltre che discovery e marker ricevono esclusivamente i campi pubblici approssimati.

La trasformazione è inoltre verificata dai test unitari descritti nella Sezione 6.2.4. Tali test hanno evidenziato un caso limite: coordinate `null` venivano normalizzate nel punto (0, 0) anziché produrre l’assenza di coordinate pubbliche. Il difetto non esponeva la posizione precisa e non violava quindi RNF-01, ma introduceva un dato geografico errato; nel flusso di geocoding poteva verificarsi se il provider principale restituiva valori nulli per entrambe le rappresentazioni delle coordinate. La normalizzazione tratta ora i valori nulli come coordinate assenti e il controllo sulla risposta del provider intercetta sia valori nulli sia valori mancanti.

6.4.3 Ricerca per prossimità

La ricerca per prossimità verifica RF-11 e parte di RF-12. L'utente autenticato deve poter indicare un'origine geografica e un raggio tra quelli supportati dal prototipo: 5 km, 10 km, 25 km e 50 km. Il sistema deve restituire soltanto libri compatibili con i vincoli di visibilità, disponibilità e distanza, ordinandoli per prossimità calcolata sulle coordinate pubbliche.

Tabella 6.10: Casi di test per ricerca per prossimità

ID	Scenario	Risultato atteso	Esito
TC-NEAR-01	Libro pubblico e disponibile entro il raggio selezionato.	Il libro compare nella lista nearby e, se presente la mappa, viene rappresentato tramite marker.	Superato
TC-NEAR-02	Libro pubblico e disponibile esterno al raggio selezionato.	Il libro non compare nei risultati della ricerca.	Superato
TC-NEAR-03	Libro privato entro il raggio.	Il libro viene escluso dai risultati anche se geograficamente vicino.	Superato
TC-NEAR-04	Libro non disponibile entro il raggio.	Il libro viene escluso dai risultati nearby.	Superato
TC-NEAR-05	Più libri validi nello stesso raggio.	I risultati sono ordinati in base alla distanza crescente calcolata sulle coordinate pubbliche.	Superato
TC-NEAR-06	Ricerca vicino a un libro specifico.	Il libro di origine viene escluso dai risultati e usato soltanto come punto di riferimento.	Superato

Per i libri persistiti, il comportamento atteso deriva dalle query **PostGIS** basate su **ST_DWithin** e **ST_Distance**. La validazione deve quindi controllare sia la lista testuale sia la coerenza del comportamento rispetto ai filtri applicati: visibilità pubblica, disponibilità diversa da non disponibile e presenza di coordinate pubbliche. L'eventuale fallback sui dati dimostrativi non persistiti deve produrre un ordinamento coerente,

pur usando il calcolo in **TypeScript** invece delle funzioni spaziali del database.

La validazione su database locale ha usato un insieme controllato di libri con coordinate pubbliche note. La query spaziale ha restituito soltanto i due libri pubblici e disponibili entro 5 km dall'origine, ordinati a 0,139 km e 1,394 km; sono stati esclusi il libro fuori raggio, quello privato, quello non disponibile, quello privo di coordinate pubbliche e il libro usato come origine.

6.4.4 Visualizzazione cartografica

La validazione della mappa riguarda la presentazione dei risultati nearby e l'usabilità dell'interazione cartografica. La mappa non deve introdurre un livello di dettaglio maggiore rispetto alla lista: marker, cluster e popup devono basarsi sulle stesse coordinate pubbliche usate dalla ricerca. In caso contrario, la visualizzazione vanificherebbe la separazione tra posizione interna e posizione esposta.

Tabella 6.11: Casi di test per visualizzazione cartografica

ID	Scenario	Risultato atteso	Esito
TC-MAP-01	Visualizzazione dei marker dei libri nearby.	I marker sono costruiti a partire da <code>publicLatitude</code> e <code>publicLongitude</code> .	Superato
TC-MAP-02	Interazione con marker, popup, cluster e controlli 2D/3D.	La mappa permette esplorazione, apertura popup, espansione cluster e cambio modalità senza perdere coerenza con la lista.	Superato
TC-MAP-03	Utilizzo su viewport mobile o con preferenza di moto ridotto.	Il comportamento viene alleggerito, evitando rotazione automatica iniziale o animazioni non necessarie.	Superato

La componente cartografica deve essere valutata anche dal punto di vista della degradazione controllata. Un errore di caricamento della mappa non deve compromettere la possibilità di consultare la lista testuale dei risultati, che resta la rappresentazione funzionale primaria. Questa distinzione è coerente con la natura accessoria della visualizzazione cartografica rispetto al requisito principale di discovery.

L'ispezione del componente **NearbyMap** ha confermato che i punti ricevuti sono trasformati in feature GeoJSON usando le coordinate pubbliche, che i risultati nearby sono gestiti tramite layer clusterizzati e popup, e che i controlli consentono pausa della rotazione, reset della camera e cambio 2D/3D. Il comportamento iniziale viene alleggerito su viewport compatti o in presenza di preferenza *reduced motion*.

6.5 Tracciabilità e sintesi della validazione

La tracciabilità conclusiva collega requisiti, casi d'uso e prove definite nel capitolo. La matrice distingue tra controlli automatici, scenari funzionali verificati e aspetti rimasti parziali perché dipendenti da condizioni esterne non rieseguite nella sessione locale.

Tabella 6.12: Matrice di tracciabilità della validazione

Requisito	Caso d'uso	Caso di test	Esito
RF-01	UC-01	TC-AUTH-01, TC-AUTH-02, VQ-08	Superato
RF-02	UC-01	TC-AUTH-03, TC-AUTH-04, VQ-08	Superato
RF-03	UC-02	TC-SEARCH-02, TC-REQ-03, VQ-08	Superato
RF-04	UC-03	Modifica profilo e avatar	Superato
RF-05	UC-04	TC-BOOK-01, TC-BOOK-02, VQ-08	Superato
RF-06	UC-04	Controllo caricamento immagini e thumbnail	Superato
RF-07	UC-05	TC-BOOK-02, TC-BOOK-03	Superato
RF-08	UC-04	TC-CAT-01, TC-CAT-02, TC-CAT-03, TC-CAT-04, TC-CAT-05	Superato
RF-09	UC-06	TC-BOOK-04, TC-SEARCH-01	Superato
RF-10	UC-06	TC-SEARCH-01	Superato
RF-11	UC-07	TC-NEAR-01, TC-NEAR-02, TC-NEAR-05	Superato

Requisito	Caso d'uso	Caso di test	Esito
RF-12	UC-08	TC-MAP-01, TC-MAP-02, TC-MAP-03	Superato
RF-13	UC-09	TC-REQ-01, TC-REQ-02, TC-REQ-03, TC-REQ-04, VQ-08	Superato
RF-14	UC-10, UC-11	TC-REQ-05, VQ-08 e controllo cancellazione richiesta pending	Superato
RF-15	UC-09	TC-REQ-02	Superato
RF-16	UC-12	Accesso alla dashboard personale	Superato
RF-17	UC-13	Accesso alla dashboard amministrativa con ruolo admin	Superato
RF-18	UC-03	Salvataggio impostazioni applicative	Superato
RNF-01	UC-07, UC-08	TC-PRIV-02, TC-MAP-01	Superato
RNF-02	UC-07, UC-08	TC-PRIV-01, TC-PRIV-03, VQ-06	Superato
RNF-03	UC-01, UC-02	TC-AUTH-01, TC-AUTH-03, TC-AUTH-04, VQ-06, VQ-07	Superato
RNF-04	UC-03–UC-13	TC-REQ-02, TC-REQ-03, TC-REQ-05, TC-BOOK-02, TC-BOOK-03, VQ-07	Superato
RNF-05	UC-01–UC-11	VQ-01, VQ-06, TC-AUTH-02, TC-BOOK-05, TC-CAT-03, TC-CAT-05	Superato
RNF-06	Trasversale	VQ-02, VQ-03	Superato
RNF-07	Trasversale	VQ-02, VQ-03, VQ-04	Superato
RNF-08	Trasversale	VQ-04 e verifica ambiente locale	Parziale: VQ-04 superato
RNF-09	Trasversale	TC-GEO-02, TC-GEO-03	Superato
RNF-10	UC-08	TC-MAP-03	Superato
RNF-11	Trasversale	VQ-02, VQ-03	Superato
RNF-12	UC-12, UC-13	Controllo dashboard personale e amministrativa	Superato

La matrice mostra che la validazione automatica copre in modo diretto gli aspetti statici, architetturali e di build, la logica pura verificata dai test unitari e, per RNF-04, i vincoli di proprietà su libri e richieste verificati dai test di integrazione e, limitatamente al percorso critico, il flusso applicativo completo verificato dallo smoke test end-to-end, mentre i requisiti funzionali di dominio sono stati verificati tramite sessioni manuali e controlli tecnici mirati. Questa distinzione è intenzionale: type checking, linting, Biome e build dimostrano la coerenza tecnica del progetto, ma la soddisfazione degli scenari d'uso richiede anche l'osservazione dei flussi di registrazione, catalogazione, richieste, discovery geografica e mappa.

Gli elementi ancora parziali non invalidano il nucleo della validazione, ma delimitano l'interpretazione dei risultati. La chiamata live a Geoapify non è stata rieseguita nella sessione documentata, pur essendo presente il ramo applicativo del provider principale; inoltre RNF-08 rimane coperto dalla build locale ma non da una verifica completa di deploy produttivo. Questi aspetti rappresentano quindi punti residui di verifica operativa, distinti dai requisiti funzionali principali che risultano coperti dalla sessione di validazione.

Capitolo 7

Risultati e discussione

7.1 Risultati ottenuti

Il lavoro svolto ha portato alla realizzazione di un prototipo web funzionante per la gestione e la condivisione di collezioni librerie private. Il risultato principale non consiste soltanto nella presenza di un catalogo digitale, ma nell'integrazione tra catalogazione, ricerca, geolocalizzazione e interazione tra utenti. Le funzionalità implementate permettono di rappresentare il libro come risorsa persistente, associarlo a un proprietario, controllarne visibilità e disponibilità, collegarlo a una posizione territoriale e abilitarne la richiesta da parte di altri utenti.

Dal punto di vista architetturale, il prototipo ha raggiunto l'obiettivo di separare le responsabilità principali del sistema. L'applicazione Next.js è organizzata per route e feature, mentre le funzionalità riutilizzabili sono collocate in pacchetti condivisi del monorepo. Tale organizzazione ha reso possibile isolare componenti con responsabilità diverse: accesso al database, modello geografico, catalogazione assistita, traduzioni e logica applicativa della web app. La verifica statica e la build di produzione hanno confermato che questa struttura è compilabile e coerente con la configurazione corrente del workspace.

Sul piano funzionale, il prototipo copre i flussi centrali individuati nell'analisi dei requisiti. Sono presenti registrazione con verifica email, autenticazione, gestione del profilo, creazione e modifica dei libri, gestione delle immagini, catalogazione assistita, consultazione del catalogo pubblico, ricerca testuale, ricerca geografica, visualizzazione cartografica, richieste tra utenti, dashboard personale e dashboard amministrativa. La copertura implementativa di tali funzionalità è stata documentata nel Capitolo 5; la validazione ha combinato controlli automatici, sessioni manuali e verifiche tecniche

mirate.

Un risultato rilevante riguarda la progettazione della posizione. Il sistema non usa la geolocalizzazione come semplice attributo accessorio del libro, ma come parte della logica di discovery. La posizione precisa e la posizione pubblica approssimata sono mantenute distinte, e le funzionalità di ricerca territoriale e mappa sono progettate per utilizzare le coordinate pubbliche. Questo risultato risponde al vincolo emerso nell'analisi: rendere utile la prossimità geografica senza trasformare l'indirizzo del proprietario in un dato pubblicamente esposto con precisione eccessiva.

La validazione tecnica eseguita ha prodotto esiti positivi per type checking, linting, controllo Biome e build di produzione. A questi risultati si sono aggiunte prove funzionali sui flussi principali: conferma email, gestione dei libri, catalogo, catalogazione assistita, richieste, dashboard, ricerca per prossimità e mappa. La validazione non equivale comunque a una certificazione produttiva: alcuni aspetti dipendenti da provider o deployment rimangono parziali e la copertura non sostituisce una suite automatizzata completa.

7.2 Discussione delle scelte progettuali

La scelta più significativa del progetto riguarda il bilanciamento tra utilità geografica e tutela della posizione. Per una piattaforma basata su beni fisici, la distanza è un'informazione utile: un libro disponibile nella stessa area dell'utente ha un valore pratico maggiore rispetto a un libro lontano. Tuttavia, esporre coordinate precise o indirizzi completi introdurrebbe un rischio non giustificato rispetto agli obiettivi del prototipo. L'approssimazione delle coordinate pubbliche rappresenta quindi un compromesso: riduce la precisione spaziale, ma conserva una granularità sufficiente per ordinare e visualizzare risultati territorialmente vicini.

Questo compromesso ha una conseguenza progettuale importante. Il sistema non cerca di garantire anonimato geografico assoluto, perché tale obiettivo richiederebbe tecniche, modelli di minaccia e verifiche più avanzate. L'obiettivo perseguito è più circoscritto e verificabile: evitare che le funzionalità di discovery usino direttamente il dato preciso. In un prototipo universitario questa distinzione è essenziale, perché permette di documentare una misura concreta di minimizzazione senza attribuirle proprietà superiori a quelle effettivamente implementate.

La seconda scelta rilevante è l'adozione di un monorepo modulare. Per un prototipo di dimensioni contenute, questa struttura introduce un costo iniziale mag-

giore rispetto a una singola applicazione monolitica: occorre mantenere pacchetti, configurazioni e confini tra aree del codice. Il beneficio è però la maggiore chiarezza architetturale. Le logiche di geocoding, persistenza, catalogazione assistita e traduzione non sono disperse nelle pagine della web app, ma collocate in aree riutilizzabili. Questo rende più esplicito il rapporto tra requisiti non funzionali di modularità e organizzazione concreta del codice.

Anche l'uso di servizi esterni è stato trattato come scelta progettuale e non come dipendenza assoluta. Geocoding, recupero metadati, OCR, storage delle immagini e invio email possono dipendere da provider o credenziali. Il prototipo prevede comportamenti degradati o fallback, così da rimanere eseguibile in ambiente locale e da non confondere la disponibilità di un servizio esterno con la correttezza del dominio applicativo. Questa scelta migliora la riproducibilità, ma impone di distinguere con attenzione tra funzionalità sempre disponibili e funzionalità condizionate dalla configurazione.

La catalogazione assistita introduce un ulteriore trade-off. Il recupero automatico di metadati tramite ISBN e l'OCR possono ridurre il carico di compilazione per l'utente, ma non garantiscono necessariamente dati completi o corretti. Per questo il prototipo mantiene il controllo umano sul form: i dati suggeriti assistono la compilazione, ma non sostituiscono la responsabilità dell'utente nella conferma delle informazioni. Questa impostazione è coerente con la natura bibliografica del dominio, nel quale piccoli errori su titolo, autore o ISBN possono compromettere la qualità del catalogo.

7.3 Limiti del prototipo

Il primo limite è metodologico e riguarda la validazione. Il repository dispone di controlli automatici di tipo statico e di build, di una suite di test unitari limitata alla logica pura di test di integrazione circoscritti ai vincoli di proprietà su libri e richieste e di un unico smoke test end-to-end sul percorso critico, eseguito localmente e non ancora integrato nella pipeline, ma non di una suite end-to-end che copra gli altri flussi e i casi negativi. Di conseguenza, gli esiti positivi ottenuti non possono essere interpretati come prova esaustiva del comportamento applicativo. I casi funzionali descritti nel Capitolo 6 documentano una sessione controllata e ripetibile, ma restano dipendenti dall'osservazione manuale e da dati di prova locali.

Un secondo limite riguarda la dipendenza da servizi esterni. La disponibilità

e la qualità dei risultati di geocoding, metadati bibliografici e OCR dipendono da provider e configurazioni non interamente controllate dal prototipo. I fallback riducono la fragilità operativa, ma non eliminano la variabilità dei risultati. In particolare, provider geografici diversi possono restituire coordinate differenti per lo stesso indirizzo, e provider bibliografici esterni possono non coprire tutti gli ISBN inseriti dagli utenti.

La tutela geografica adottata è intenzionalmente semplice. L'arrotondamento delle coordinate pubbliche riduce la granularità del dato esposto, ma non costituisce una tecnica avanzata di anonimizzazione. Non sono state implementate analisi formali del rischio di reidentificazione, strategie adattive basate sulla densità territoriale o meccanismi di aggregazione dipendenti dal contesto urbano. Il sistema quindi migliora la minimizzazione rispetto all'esposizione del punto preciso, ma non deve essere presentato come soluzione completa di privacy geografica.

Dal punto di vista funzionale, il prototipo non copre tutte le esigenze di una piattaforma produttiva. Restano fuori dal perimetro elementi come moderazione avanzata dei contenuti, messaggistica completa tra utenti, notifiche transazionali estese, gestione operativa del prestito dopo l'accettazione, procedure amministrative complesse e metriche analitiche approfondite. Tali omissioni non impediscono di valutare il nucleo progettuale, ma delimitano chiaramente il campo di validità dei risultati.

Infine, la valutazione non include dati d'uso raccolti da utenti reali. Non sono quindi disponibili misure quantitative su tempi di compilazione, soddisfazione degli utenti, precisione percepita della mappa o tasso di completamento delle richieste. La discussione dei risultati deve pertanto rimanere tecnica e progettuale, evitando conclusioni empiriche non supportate.

7.4 Sviluppi futuri

Il primo sviluppo futuro riguarda l'estensione dell'automazione della validazione funzionale. I test di integrazione introdotti per i vincoli di proprietà su libri e richieste potrebbero essere estesi agli altri flussi della traccia manuale, mentre lo smoke test end-to-end potrebbe essere integrato nella pipeline e ampliato a ricerca, geolocalizzazione e casi negativi, riducendo al contempo la dipendenza da servizi esterni come Nominatim. Ciò ridurrebbe la dipendenza dall'osservazione manuale e permetterebbe di ripetere con maggiore continuità i controlli su autenticazione, catalogazione, geolocalizzazione e richieste.

Un secondo sviluppo riguarda il rafforzamento della privacy geografica. L'approssimazione attuale potrebbe essere sostituita o affiancata da strategie più robuste, ad esempio granularità variabile in base al contesto, soglie minime di densità, visualizzazione per aree invece che per punti o politiche configurabili dall'utente. Queste evoluzioni richiederebbero però una valutazione più approfondita del rischio e dell'impatto sull'utilità della ricerca.

La piattaforma potrebbe inoltre evolvere nella gestione del ciclo di vita delle richieste. Dopo l'accettazione, il sistema potrebbe supportare accordi tra utenti, notifiche, stati successivi del prestito, conferma di restituzione e storico delle interazioni. Tali funzionalità amplirebbero il passaggio dalla semplice discovery alla gestione operativa della condivisione, ma richiederebbero anche regole più precise di responsabilità e moderazione.

Ulteriori sviluppi possono riguardare la qualità del catalogo. La catalogazione assistita potrebbe integrare più provider bibliografici, controlli di qualità sui metadati, suggerimenti per duplicati e strumenti di revisione manuale. Analogamente, l'OCR potrebbe essere valutato su casi reali e confrontato con l'inserimento manuale, evitando però di automatizzare la persistenza di dati non verificati.

Infine, una futura evoluzione produttiva dovrebbe affrontare aspetti infrastrutturali e operativi non centrali nel prototipo: deployment stabile, osservabilità, backup, protezioni contro gli abusi a livello di rete in aggiunta ai limiti applicativi già presenti, gestione sicura dei segreti, politiche di moderazione e documentazione completa delle procedure amministrative. Questi interventi non modificano il nucleo concettuale del progetto, ma sarebbero necessari per trasformare il prototipo in un servizio utilizzabile in modo continuativo.

Capitolo 8

Conclusioni

8.1 Sintesi del lavoro svolto

La tesi ha affrontato il problema della scarsa accessibilità del patrimonio librario privato, considerando il libro fisico come una risorsa distribuita che può acquisire valore collettivo se descritta, resa ricercabile e collegata a un canale di interazione. Il lavoro non ha perseguito la digitalizzazione dei contenuti, ma la rappresentazione informativa delle copie possedute dagli utenti e la loro scoperta controllata tramite criteri bibliografici e geografici.

Il percorso seguito ha collegato analisi, progettazione, implementazione e validazione. Dopo aver individuato attori, casi d'uso e requisiti, il progetto ha definito un'architettura web modulare, un modello dati coerente con il dominio applicativo e una strategia di gestione della posizione basata sulla separazione tra coordinate interne e coordinate pubbliche approssimate. L'implementazione ha poi tradotto tali scelte in un prototipo basato su applicazione web, database relazionale con supporto geospaziale, autenticazione, gestione dei libri, ricerca, mappa e richieste tra utenti.

Gli obiettivi iniziali risultano raggiunti nella misura propria di un prototipo universitario. La piattaforma consente di catalogare libri appartenenti a utenti privati, pubblicarli con visibilità controllata, renderli ricercabili tramite catalogo e ricerca territoriale, visualizzarli su mappa e avviare richieste di consultazione, prestito o informazione. La validazione ha confermato la correttezza statica, la producibilità della build e il comportamento dei principali flussi funzionali, pur lasciando fuori dal perimetro una certificazione produttiva completa.

8.2 Contributo progettuale

Il contributo principale del lavoro è la progettazione e realizzazione di una piattaforma geolocalizzata orientata alla condivisione controllata di risorse librerie fisiche. L'aspetto informatico rilevante non risiede in una singola tecnologia, ma nell'integrazione di più responsabilità: gestione degli account, persistenza del catalogo, ricerca bibliografica, discovery geografica, tutela della posizione e workflow di richiesta.

La modellazione del dominio ha permesso di distinguere le entità centrali del sistema: utenti, libri, immagini, posizioni, statistiche e richieste. Questa separazione rende espliciti i vincoli di proprietà e autorizzazione, evitando che il catalogo venga trattato come un semplice elenco pubblico privo di regole operative. Ogni libro resta associato a un proprietario, ogni richiesta collega richiedente e proprietario, e le operazioni protette richiedono una sessione coerente con il ruolo dell'utente.

Un secondo contributo riguarda la gestione della prossimità geografica. Il sistema dimostra come un requisito di localizzazione possa essere integrato in una piattaforma di catalogazione senza esporre direttamente la posizione precisa. La scelta di usare coordinate pubbliche approssimate nelle funzioni di discovery rappresenta una soluzione semplice ma coerente con il principio di minimizzazione: il dato geografico viene usato per rendere utile la ricerca, non per pubblicare l'indirizzo del proprietario.

Il terzo contributo è architetturale. L'organizzazione modulare del repository separa l'applicazione web dalle parti riutilizzabili e rende più chiara la corrispondenza tra requisiti e componenti tecnici. Questa impostazione facilita la manutenzione del prototipo, consente di isolare le dipendenze esterne e rende più controllabile l'evoluzione futura del sistema.

8.3 Considerazioni finali

Il risultato ottenuto mostra che la condivisione di libri appartenenti a collezioni private può essere affrontata tramite una piattaforma web che combina catalogazione, ricerca e interazione tra utenti. La dimensione geografica aggiunge utilità pratica, perché collega la disponibilità del libro alla possibilità concreta di raggiungerlo; allo stesso tempo, richiede scelte progettuali attente per non trasformare la posizione in un'informazione esposta in modo eccessivo.

Il prototipo mantiene consapevolmente un perimetro limitato. Non pretende di essere una piattaforma produttiva completa, né di fornire una soluzione definitiva ai

problemi di privacy geografica. La sua utilità sta nel dimostrare un modello realizzabile e verificabile, nel quale la discovery territoriale viene integrata con autenticazione, proprietà dei contenuti, richieste e approssimazione delle coordinate. I limiti individuati non annullano il contributo del lavoro, ma ne definiscono il campo di validità.

Le evoluzioni future più rilevanti riguardano l'automazione dei test, la verifica in ambiente di deploy, il rafforzamento delle tecniche di tutela geografica e l'estensione del ciclo di vita delle richieste. Tali sviluppi permetterebbero di passare da un prototipo accademico a una piattaforma più robusta, mantenendo però il principio centrale emerso dal progetto: rendere visibili e condivisibili risorse fisiche distribuite senza rinunciare al controllo da parte degli utenti che le possiedono.

Ringraziamenti

Bibliografia

- [1] Hongbo Jiang et al. «Location Privacy-preserving Mechanisms in Location-based Services: A Comprehensive Survey». In: *ACM Computing Surveys* 54.1 (2021), pp. 1–36. DOI: 10.1145/3423165.
- [2] Hira Rasheed et al. «Preserving Location-Query Privacy in Location-Based Services: A Review». In: *Security and Privacy* 7.5 (2024), e412. DOI: 10.1002/spy2.412.
- [3] LibraryThing. *LibraryThing*. URL: <https://www.librarything.com/> (visitato il giorno 30/08/2026).
- [4] LibraryThing. *LibraryThing Local*. URL: <https://www.librarything.com/local> (visitato il giorno 30/08/2026).
- [5] Libib. *Libib*. URL: <https://www.libib.com/> (visitato il giorno 30/08/2026).
- [6] BookCrossing. *BookCrossing*. URL: <https://www.bookcrossing.com/> (visitato il giorno 30/08/2026).
- [7] Little Free Library. *Little Free Library*. URL: <https://littlefreelibrary.org/> (visitato il giorno 30/08/2026).
- [8] Simge Özdal Oktay, Sven Heitmann e Christian Kray. «Linking Location Privacy, Digital Sovereignty and Location-Based Services: A Meta Review». In: *Journal of Location Based Services* 18.1 (2024), pp. 1–52. DOI: 10.1080/17489725.2023.2239180.
- [9] Vercel. *Next.js Documentation*. URL: <https://nextjs.org/docs> (visitato il giorno 30/08/2026).
- [10] Nx. *Nx Documentation*. URL: <https://nx.dev/docs> (visitato il giorno 30/08/2026).
- [11] PostgreSQL Global Development Group. *PostgreSQL Documentation*. URL: <https://www.postgresql.org/docs/> (visitato il giorno 30/08/2026).

- [12] PostGIS Project Steering Committee. *PostGIS Documentation*. URL: <https://postgis.net/documentation/> (visitato il giorno 30/08/2026).
- [13] Prisma Data, Inc. *Prisma Documentation*. URL: <https://www.prisma.io/docs> (visitato il giorno 30/08/2026).
- [14] Auth.js. *Auth.js Documentation*. URL: <https://authjs.dev/> (visitato il giorno 30/08/2026).
- [15] MapLibre. *MapLibre GL JS Documentation*. URL: <https://maplibre.org/maplibre-gl-js/docs/> (visitato il giorno 30/08/2026).
- [16] Internet Archive. *Open Library*. URL: <https://openlibrary.org/> (visitato il giorno 30/08/2026).
- [17] Cloudflare. *Cloudflare Workers Documentation*. URL: <https://developers.cloudflare.com/workers/> (visitato il giorno 30/08/2026).